Part A: A1. Natural Language Processing.

Natural language Processing is a branch of artificial intelligence that enables computers to understand, process and generate human languages. It combines linguistic and ML to perform tasks involving text and speech.

Application	Input	Out put
1- Sentiment Analysis.	"This movie is amazing"	Postive Review.
2- Machine Translation.	"How are you?"	"آپ کیسے ہیں؟"
3- Spam Detection"	"You won Prize"	Spam or junk classification.

A2: Three Eras of NLP.

Rule Based era: This era used manually written linguistic rule, dictionaries and grammer to process language.
Land mark System: ELIZA (1966) an early chatbot for predefined responses.

Statistical Era: This era used probabilities and Statistical models learned from text data to predict languag Patterns. **LandMark method:** Hidden Markov Models widely used for tasks such as part of speech tagging.

Neural Era: The era uses artificial neural network to learn complex language pattern from large datasets.
Landmark Paper: Attention all you need (2017), which introduced the Transformer architecutred used in NLP.

```
PS D:\NLP-assignment1> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
PS D:\NLP-assignment1> .\nlp-env\Scripts\Activate.ps1
(nlp-env) PS D:\NLP-assignment1> python -m pip install numpy pandas nltk spacy scikit-learn jupyter ipykernel
```

```
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
(nlp-env) PS D:\NLP-assignment1> python -c "import nltk; nltk.download(['punkt_tab', 'stopwords', 'wordnet', 'omw-1.4'])"

[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\hp\AppData\Roaming\nltk_data...
[nltk_data] Downloading package omw-1.4 to
[nltk_data]     C:\Users\hp\AppData\Roaming\nltk_data...
(nlp-env) PS D:\NLP-assignment1>
```

In [1]:
```python
import sys
import numpy as np
import pandas as pd
import nltk
import spacy
import sklearn

print("Python Version:", sys.version.split()[0])
print("NumPy Version:", np.__version__)
print("Pandas Version:", pd.__version__)
print("NLTK Version:", nltk.__version__)
print("spaCy Version:", spacy.__version__)
print("Scikit-learn Version:", sklearn.__version__)
```

```
Python Version: 3.12.4
NumPy Version: 2.5.4
Pandas Version: 3.0.6
NLTK Version: 3.10.3
spaCy Version: 3.8.16
Scikit-learn Version: 1.9.1
```

Why Is a Virtual Environment Better Than Global Installation?

A virtual environment creates an isolated space for each Python project, allowing us to install libraries without affecting the global Python installation. It prevents dependency and version conflicts between different projects. It also makes projects easier to manage, share, and reproduce.

B1

In [3]:

```
import string
from collections import Counter

D1 = "The cat sat on the mat."
D2 = "The dog chased the cat."
D3 = "Dogs and cats are friendly pets."

def word_freq(text):

    text = text.lower()

    text = text.translate(str.maketrans("", "", string.punctuation))

    words = text.split()

    freq = {}

    for word in words:
        if word in freq:
            freq[word] += 1
        else:
            freq[word] = 1

    return freq



combined_text = D1 + " " + D2 + " " + D3

frequencies = word_freq(combined_text)

print("Word Frequencies:")
print(frequencies)

top_3 = Counter(frequencies).most_common(3)

print("\nTop 3 Most Frequent Words:")

for word, count in top_3:
    print(word, ":", count)
```

```
Word Frequencies:
{'the': 4, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1, 'dog': 1, 'chased': 1, 'dogs': 1, 'an
d': 1, 'cats': 1, 'are': 1, 'friendly': 1, 'pets': 1}

Top 3 Most Frequent Words:
the : 4
cat : 2
sat : 1
```

Explanation of Results

The word_freq() function successfully calculates the frequency of each word in the combined documents. The function uses a Python dictionary for counting word occurrences without using Counter inside the function. The Counter.most_common(3) method is used outside the function to find the three most frequent words. stemming and lemmatization is not applied.

B2

In [4]:

```
a = np.array([1, 2, 3])
b = np.array([2, 0, 1])

dot_product = np.dot(a, b)

# Euclidean (L2) norm of each vector
norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)

# cosine similarity
cosine_similarity = dot_product / (norm_a * norm_b)

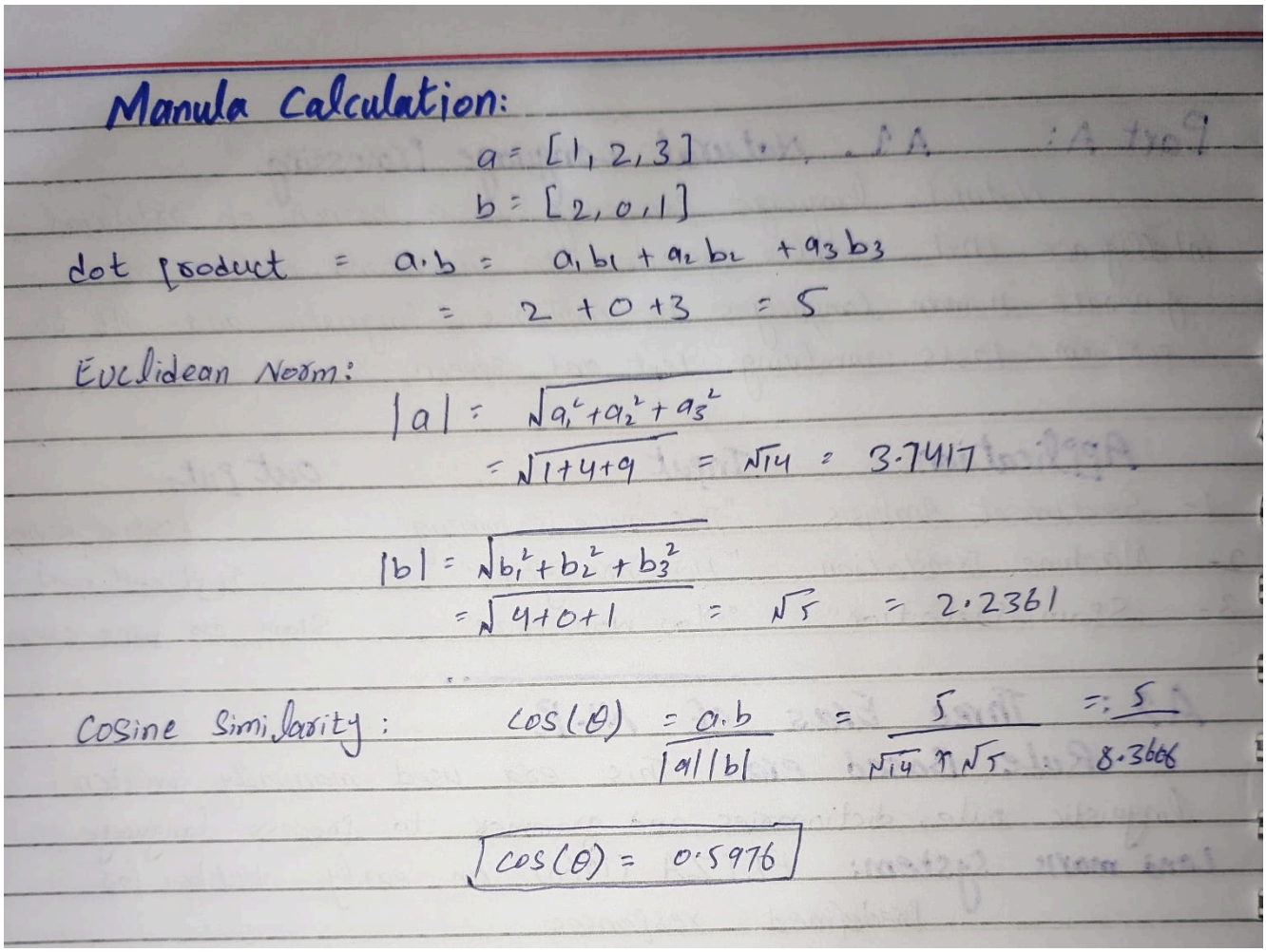
print("Vector a:", a)
print("Vector b:", b)

print("\nDot Product:", dot_product)
print("L2 Norm of a:", round(norm_a, 4))
print("L2 Norm of b:", round(norm_b, 4))
print("Cosine Similarity:", round(cosine_similarity, 4))
```

```
Vector a: [1 2 3]
Vector b: [2 0 1]

Dot Product: 5
L2 Norm of a: 3.7417
L2 Norm of b: 2.2361
Cosine Similarity: 0.5976
```

Calculation of cosine similarity on paper



Result and Verification

The dot product of vectors a and b is **5** while their Euclidean norms are **3.7417** and **2.2361** respectively. The cosine similarity calculated using NumPy is **0.5976**which matches the manually calculated value of **0.5976**. both methods produce the same result confirming that the calculations are correct.

B3

In [5]:
```
D1 = "The cat sat on the mat."
D2 = "The dog chased the cat."
D3 = "Dogs and cats are friendly pets."
# DataFrame
data = {
    "doc_id": ["D1", "D2", "D3"],
    "text": [D1, D2, D3]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
display(df)

# number of words and characters
df["n_words"] = df["text"].str.split().str.len()
df["n_chars"] = df["text"].str.len()

print("DataFrame with word and character counts:")
display(df)

#documents with more than 5 words
filtered_df = df[df["n_words"] > 5]

print("Documents with More Than 5 Words:")
display(filtered_df)

# mean word count
mean_words = df["n_words"].mean()

print("Mean Number of Words:", round(mean_words, 2))
```

```
Original DataFrame:
```

	doc_id	text
0	D1	The cat sat on the mat.
1	D2	The dog chased the cat.
2	D3	Dogs and cats are friendly pets.

```
DataFrame with word and character counts:
```

	doc_id	text	n_words	n_chars
0	D1	The cat sat on the mat.	6	23
1	D2	The dog chased the cat.	5	23
2	D3	Dogs and cats are friendly pets.	6	32

```
Documents with More Than 5 Words:
```

	doc_id	text	n_words	n_chars
0	D1	The cat sat on the mat.	6	23
2	D3	Dogs and cats are friendly pets.	6	32

```
Mean Number of Words: 5.67
```

Interpretation

The mean word count is **5.67** which indicates that each document contains approximately 6 words on average.

explanation of results

- pd.dataframe() creates a table containing the document ids and their text.
- .str.split() splits each document's text into individual words.
- .str.len() counts the words after splitting and counts characters when applied directly to text.
- df[df["n_words"] > 5] selects documents containing more than 5 words.
- .mean() calculates the average number of words, which is 5.67.

C1

In [8]:
```
text = "Dr. Smith's e-mail isn't working!!! Visit https://nlp.edu now :)"

#  str.split()
python_tokens = text.split()

#NLTK
nltk_tokens = nltk.word_tokenize(text)

# spaCy
nlp = spacy.load("en_core_web_sm")
doc = nlp(text)

spacy_tokens = [token.text for token in doc]

data = {
    "Tokenizer": ["Python str.split()", "NLTK word_tokenize()", "spaCy"],
    "Tokens": [python_tokens, nltk_tokens, spacy_tokens]
}

df = pd.DataFrame(data)

print("Result of each Method in the Table :")
display(df)
```

```
Result of each Method in the Table :
```

	Tokenizer	Tokens
0	Python str.split()	[Dr., Smith's, e-mail, isn't, working!!!, Visi...
1	NLTK word_tokenize()	[Dr., Smith, 's, e-mail, is, n't, working, !, ...
2	spaCy	[Dr., Smith, 's, e, -, mail, is, n't, working,...

Explanation of Results

a word is a meaningful unit of language while a token can be a word part of a word or a symbol python str split separates text based on spaces so punctuation and contractions remain attached to words nltk and spacy separate contractions into smaller tokens and split punctuation marks from words spacy splits hyphenated words into separate tokens while nltk may keep them together because both use different tokenization rules nltk and spacy also handle urls and smiley symbols differently because their tokenization methods follow different rules

C2

In [9]:
```python
import re
# Given text
text = """Contact ali@uni.edu.pk or sara_k@mail.com.
Call 0300-1234567 or +92 321 7654321.
Follow #NLP2026 and #AI at https://nlp.example.org/course."""

# Matches email addresses containing a username domain and extension
email_pattern = r"\b[\w.-]+@[\w.-]+\.[a-zA-Z]{2,}\b"
emails = re.findall(email_pattern, text)

# Matches a hashtag followed by letters numbers or underscores
hashtag_pattern = r"#[a-zA-Z0-9_]+"
hashtags = re.findall(hashtag_pattern, text)

# Matches http or https links with a domain and optional path
url_pattern = r"https?://[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}(?:/[a-zA-Z0-9_/-]+)?"
urls = re.findall(url_pattern, text)

# Matches +92 numbers and other international numbers
phone_pattern = r"\b0\d{3}-\d{7}\b|\+92\s\d{3}\s\d{7}\b"
phones = re.findall(phone_pattern, text)

# Replaces each matched URL with the token <URL>
new_text = re.sub(url_pattern, "<URL>", text)


print("Email Addresses:", emails)
print("Hashtags:", hashtags)
print("URLs:", urls)
print("Phone Numbers:", phones)

print("\nText After URL Replacement:")
print(new_text)
```

```
Email Addresses: ['ali@uni.edu.pk', 'sara_k@mail.com']
Hashtags: ['#NLP2026', '#AI']
URLs: ['https://nlp.example.org/course']
Phone Numbers: ['0300-1234567', '+92 321 7654321']

Text After URL Replacement:
Contact ali@uni.edu.pk or sara_k@mail.com.
Call 0300-1234567 or +92 321 7654321.
Follow #NLP2026 and #AI at <URL>.
```

Incorrect Matching Examples

email pattern the pattern may incorrectly accept an email containing consecutive dots in the username such as ali..khan@gmail.com hashtag pattern the pattern may incorrectly extract a hashtag from abc#AI even though the hash symbol is attached to another word url pattern the pattern may incorrectly accept a url containing an invalid hostname such as https://-abc.com (https://-abc.com) phone number pattern the pattern may incorrectly accept 0000-1234567 even though it is not a valid pakistani mobile number

C3

In [10]:

```
def clean_text(s):
    #  lowercase
    s = s.lower()

    #  HTML tags
    s = re.sub(r"<[^>]+>", " ", s)

    # Remove URLs
    s = re.sub(r"https?://\S+", " ", s)

    # Remove punctuation
    s = s.translate(str.maketrans("", "", string.punctuation))

    # Remove digits
    s = re.sub(r"\d+", "", s)

    # Remove repeated whitespace
    s = re.sub(r"\s+", " ", s).strip()

    return s


#  input
text = """<p>I LOVED this phone!!! Best $$ I've ever spent :)</p>
Visit http://shop.example.com/deal?id=42 for 50% OFF...
It's NOT   good. I won't buy it again!!!"""

cleaned_text = clean_text(text)

print("Before Cleaning:")
print(text)

print("\nAfter Cleaning:")
print(cleaned_text)
```

```
Before Cleaning:
<p>I LOVED this phone!!! Best $$ I've ever spent :)</p>
Visit http://shop.example.com/deal?id=42 for 50% OFF...
It's NOT   good. I won't buy it again!!!

After Cleaning:
i loved this phone best ive ever spent visit for off its not good i wont buy it again
```

Explanation of Results

the clean text function processes the entire input as a single string it converts the text to lowercase and removes html tags and urls punctuation marks and digits are removed from the text extra spaces and line breaks are replaced with a single space

C4

In [11]:

```
from nltk.corpus import stopwords

D1 = "The cat sat on the mat."
D2 = "The dog chased the cat."
D3 = "Dogs and cats are friendly pets."

documents = [D1, D2, D3]

#stop words from NLTK
stop_words = set(stopwords.words("english"))


def tokenize(text):
    return re.findall(r"[a-z]+(?:'[a-z]+)?", text.lower())


def remove_stopwords(text, stop_list):
    words = tokenize(text)
    filtered_words = []

    for word in words:
        if word not in stop_list:
            filtered_words.append(word)

    return filtered_words


# calculate percentages
results = []

total_tokens = 0
total_removed = 0

for i, text in enumerate(documents, 1):

    original_words = tokenize(text)
    filtered_words = remove_stopwords(text, stop_words)

    removed = len(original_words) - len(filtered_words)
    percentage = (removed / len(original_words)) * 100

    total_tokens += len(original_words)
    total_removed += removed

    results.append({
        "Document": f"D{i}",
        "Original Tokens": len(original_words),
        "Removed Tokens": removed,
        "Percentage Removed": round(percentage, 2),
        "Remaining Words": " ".join(filtered_words)
    })

#table
df = pd.DataFrame(results)

print("Stop Word Removal Results:")
display(df)

# e overall percentage
overall_percentage = (total_removed / total_tokens) * 100

print("Total Tokens:", total_tokens)
print("Total Removed:", total_removed)
print("Overall Percentage Removed:", round(overall_percentage, 2), "%")
```

```
#  removing negations
sentence = "It's NOT good"

print("\nOriginal Sentence:", sentence)

normal_result = remove_stopwords(sentence, stop_words)

print("After Stop Word Removal:", " ".join(normal_result))


# Custom stop word list that preserves negations
negations = {
    "no", "nor", "not", "never", "cannot",
    "don't", "doesn't", "didn't",
    "isn't", "aren't", "wasn't", "weren't"
}

custom_stop_words = stop_words - negations

# apply custom stop word removal to D1 to D3
print("\nCustom Stop Word Removal:")

for i, text in enumerate(documents, 1):
    filtered = remove_stopwords(text, custom_stop_words)
    print(f"D{i}:", " ".join(filtered))


custom_result = remove_stopwords(sentence, custom_stop_words)

print("\nOriginal Sentence:", sentence)
print("With Standard Stop Words:", " ".join(normal_result))
print("With Custom Stop Words:", " ".join(custom_result))
```

```
Stop Word Removal Results:
```

	Document	Original Tokens	Removed Tokens	Percentage Removed	Remaining Words
0	D1	6	3	50.00	cat sat mat
1	D2	5	2	40.00	dog chased cat
2	D3	6	2	33.33	dogs cats friendly pets

```
Total Tokens: 17
Total Removed: 7
Overall Percentage Removed: 41.18 %

Original Sentence: It's NOT good
After Stop Word Removal: good

Custom Stop Word Removal:
D1: cat sat mat
D2: dog chased cat
D3: dogs cats friendly pets

Original Sentence: It's NOT good
With Standard Stop Words: good
With Custom Stop Words: not good
```

Explanation of Result

the nltk stopwords list removes common words such as the on and are from the documents a total of 17 tokens were processed and 7 tokens were removed resulting in 41 point 18 percent stopword removal the custom stopword list produces the same results for the three documents because they do not contain any negations the example sentence shows that removing negations can change the original meaning of the text

C5

In [29]:

```
from nltk.stem import PorterStemmer, SnowballStemmer, WordNetLemmatizer

words = [
    "studies",
    "studying",
    "better",
    "running",
    "was",
    "geese",
    "happily",
    "universities"
]

#  parts of speech for lemmatization
pos_tags = ["n", "v", "a", "v", "v", "n", "r", "n"]

porter = PorterStemmer()
snowball = SnowballStemmer("english")
lemmatizer = WordNetLemmatizer()

results = []

#  stemming and lemmatization
for word, pos in zip(words, pos_tags):

    porter_stem = porter.stem(word)
    snowball_stem = snowball.stem(word)
    lemma = lemmatizer.lemmatize(word, pos=pos)

    results.append({
        "Word": word,
        "Porter Stem": porter_stem,
        "Snowball Stem": snowball_stem,
        "Lemma": lemma
    })

# dataFrame
df = pd.DataFrame(results)

print("Normalization Results:")
display(df)
```

```
Normalization Results:
```

	Word	Porter Stem	Snowball Stem	Lemma
0	studies	studi	studi	study
1	studying	studi	studi	study
2	better	better	better	good
3	running	run	run	run
4	was	wa	was	be
5	geese	gees	gees	goose
6	happily	happili	happili	happily
7	universities	univers	univers	university

Incorrect Stemming Examples

studies is reduced to studi by both stemmers which is not a valid english word while lemmatization produces study universities is reduced to univers by both stemmers which is not a complete english word while lemmatization produces university

Stemming vs Lemmatization

stemming is preferred in tasks like information retrieval and search engines where fast text normalization is needed lemmatization is preferred in tasks like text analysis and natural language understanding where meaningful dictionary forms are important stemming is generally faster but can produce incorrect word forms while lemmatization is more linguistically accurate because it considers the part of speech

D1

In [30]:

```
D1 = "The cat sat on the mat."
D2 = "The dog chased the cat."
D3 = "Dogs and cats are friendly pets."

text = D1 + " " + D2 + " " + D3

# lowercase
text = text.lower()

# Remove punctuation
text = text.translate(str.maketrans("", "", string.punctuation))

# Split text into words
words = text.split()

# Create vocabulary in alphabetical order
vocabulary = sorted(set(words))

print("Vocabulary:", vocabulary)
print("Vocabulary Size:", len(vocabulary))

# Function to create one-hot vectors
def one_hot(word, vocabulary):

    vector = np.zeros(len(vocabulary), dtype=int)

    index = vocabulary.index(word)

    vector[index] = 1

    return vector


cat_vector = one_hot("cat", vocabulary)
dog_vector = one_hot("dog", vocabulary)
pets_vector = one_hot("pets", vocabulary)

print("\nOne-Hot Vectors:")
print("cat :", cat_vector)
print("dog :", dog_vector)
print("pets:", pets_vector)

dot_product = np.dot(cat_vector, dog_vector)

print("\nDot Product of cat and dog:", dot_product)
```

```
Vocabulary: ['and', 'are', 'cat', 'cats', 'chased', 'dog', 'dogs', 'friendly', 'mat',
'on', 'pets', 'sat', 'the']
Vocabulary Size: 13

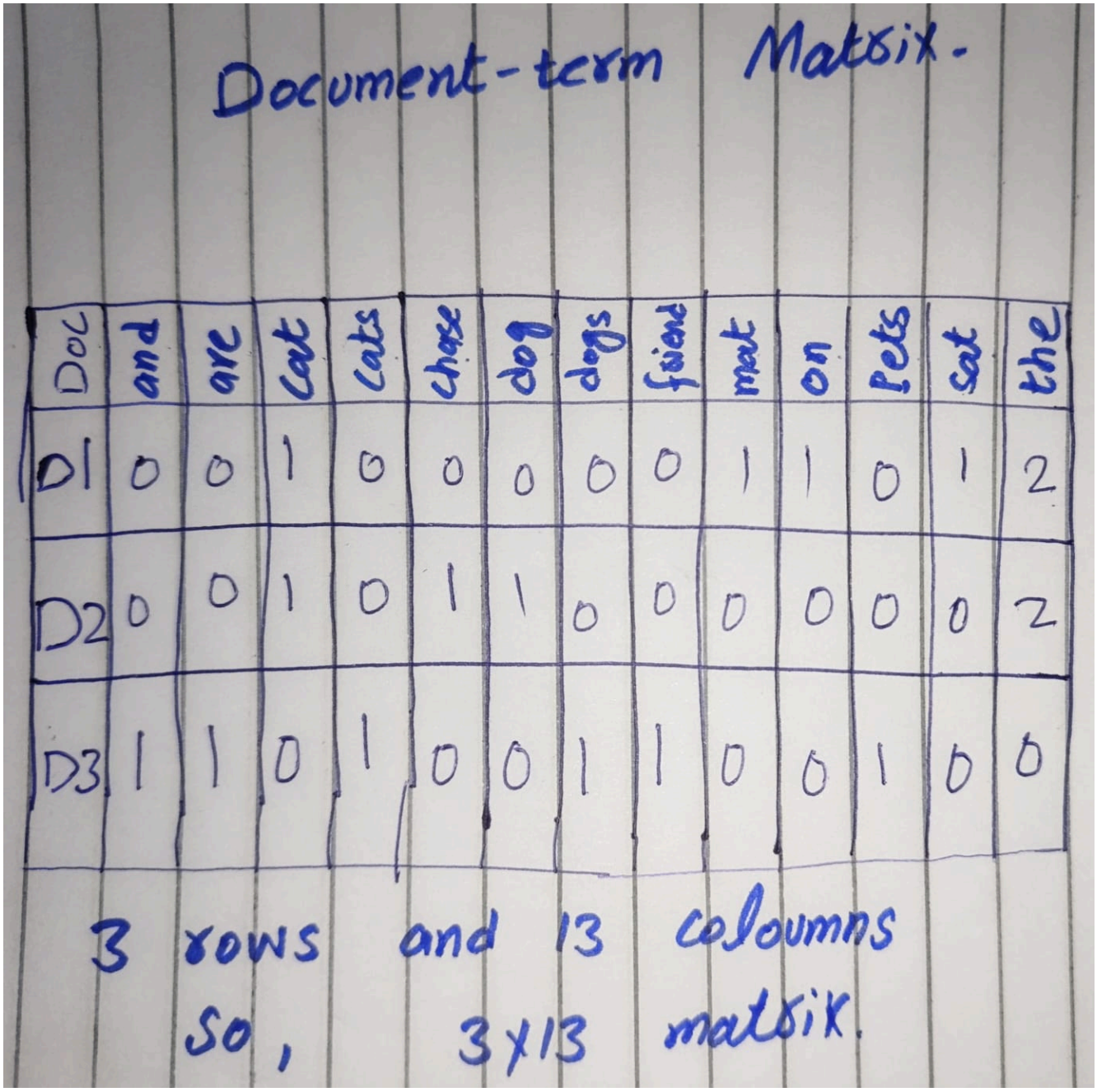
One-Hot Vectors:
cat : [0 0 1 0 0 0 0 0 0 0 0 0 0]
dog : [0 0 0 0 0 1 0 0 0 0 0 0 0]
pets: [0 0 0 0 0 0 0 0 0 0 1 0 0]

Dot Product of cat and dog: 0
```

Explanation of Results

the vocabulary contains 13 unique words arranged in alphabetical order each word is represented by a vector of 13 dimensions containing one value of 1 and all remaining values of 0 the dot product of cat and dog is 0 because their active positions are different this shows that one hot encoding cannot capture semantic similarity between different words even when their meanings are related another limitation is that one hot encoding produces high dimensional sparse vectors which require more memory as the vocabulary grows

D2



In [32]:

```
from sklearn.feature_extraction.text import CountVectorizer

# Define the three documents
D1 = "The cat sat on the mat."
D2 = "The dog chased the cat."
D3 = "Dogs and cats are friendly pets."

documents = [D1, D2, D3]

# Create CountVectorizer object
vectorizer = CountVectorizer()

# Convert documents into Bag of Words matrix
bow_matrix = vectorizer.fit_transform(documents)

vocabulary = vectorizer.get_feature_names_out()

#  matrix into DataFrame
bow_df = pd.DataFrame(
    bow_matrix.toarray(),
    columns=vocabulary,
    index=["D1", "D2", "D3"]
)

print("Vocabulary:", list(vocabulary))
print("Vocabulary Size:", len(vocabulary))

print("\nBag of Words Matrix:")
display(bow_df)


print("\n No there is no difference between the manually calculated and this matrix !")
```

```
Vocabulary: ['and', 'are', 'cat', 'cats', 'chased', 'dog', 'dogs', 'friendly', 'mat',
'on', 'pets', 'sat', 'the']
Vocabulary Size: 13

Bag of Words Matrix:
```

	and	are	cat	cats	chased	dog	dogs	friendly	mat	on	pets	sat	the
D1	0	0	1	0	0	0	0	0	1	1	0	1	2
D2	0	0	1	0	1	1	0	0	0	0	0	0	2
D3	1	1	0	1	0	0	1	1	0	0	1	0	0

```
 No there is no difference between the manually calculated and this matrix !
```

In [41]:

```
binary_vectorizer = CountVectorizer(binary=True)
binary_matrix = binary_vectorizer.fit_transform(documents)

binary_vocabulary = binary_vectorizer.get_feature_names_out()

binary_df = pd.DataFrame(
    binary_matrix.toarray(),
    columns=binary_vocabulary,
    index=["D1", "D2", "D3"]
)

print("Binary Bag of Words Matrix:")
display(binary_df)
```

```
Binary Bag of Words Matrix:
```

	and	are	cat	cats	chased	dog	dogs	friendly	mat	on	pets	sat	the
D1	0	0	1	0	0	0	0	0	1	1	0	1	1
D2	0	0	1	0	1	1	0	0	0	0	0	0	1
D3	1	1	0	1	0	0	1	1	0	0	1	0	0

Explanation of Results

binary bag of words records only whether a word exists in a document unlike regular bag of words it does not count repeated occurrences binary counts are preferable when the presence of a word is more important than its frequency for example in spam detection the presence of certain suspicious words may be useful even if they appear only once binary bag of words can also reduce the influence of frequently repeated words

D3

TF-IDF

D1 : the cat sat on mat
D2 : the dog chased the cat
D3: dogs and cats are friendly Pets.

Total documents = N = 3
Total words in D2 = 5

TF

TF (the, D2) = 2/5 = 0.4
TF (cat, D2) = 1/5 = 0.2
TF (dog, D2) = 1/5 = 0.2

IDF

IDF (the) = ln(3/2) = 0.4055
IDF (cat) = ln(3/2) = 0.4055
IDF (dog) = ln(3/1) = 1.0986

TF-IDF

TF-IDF (the, D2) = 0.4 × 0.4055 = 0.1622
TF-IDF (cat, D2) = 0.2 × 0.4055 = 0.0811
TF-IDF (dog, D2) = 0.2 × 1.0986 = 0.2197

Why Word the Receive a low Weight.

The word that appears in two of the three documents so it receives a low idf value than the word dog common words are less useful for distinguishing documents during search and classification because they provide less information about the topic.

In [42]:

```
from sklearn.feature_extraction.text import TfidfVectorizer

D1 = "The cat sat on the mat."
D2 = "The dog chased the cat."
D3 = "Dogs and cats are friendly pets."

documents = [D1, D2, D3]

# TF-IDF vectorizer
vectorizer = TfidfVectorizer()

# TF-IDF vectors
tfidf_matrix = vectorizer.fit_transform(documents)

# Get vocabulary
vocabulary = vectorizer.get_feature_names_out()

tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=vocabulary,
    index=["D1", "D2", "D3"]
)

print("TF-IDF Matrix:")
display(tfidf_df.round(4))

terms = ["the", "cat", "dog"]
counts = [2, 1, 1]

results = []

for word, count in zip(terms, counts):

    index = list(vocabulary).index(word)

    tf = count / 5
    idf = vectorizer.idf_[index]
    tfidf = tfidf_df.loc["D2", word]

    results.append({
        "Term": word,
        "Manual TF": round(tf, 4),
        "Sklearn IDF": round(idf, 4),
        "Sklearn TF-IDF": round(tfidf, 4)
    })

comparison_df = pd.DataFrame(results)

print("\nTF-IDF Values for D2:")
display(comparison_df)
```

TF-IDF Matrix:

	and	are	cat	cats	chased	dog	dogs	friendly	mat	on	pets	sat	th
D1	0.0000	0.0000	0.3133	0.0000	0.0000	0.0000	0.0000	0.0000	0.412	0.412	0.0000	0.412	0.626
D2	0.0000	0.0000	0.3439	0.0000	0.4521	0.4521	0.0000	0.0000	0.000	0.000	0.0000	0.000	0.687
D3	0.4082	0.4082	0.0000	0.4082	0.0000	0.0000	0.4082	0.4082	0.000	0.000	0.4082	0.000	0.000

TF-IDF Values for D2:

	Term	Manual TF	Sklearn IDF	Sklearn TF-IDF
0	the	0.4	1.2877	0.6877
1	cat	0.2	1.2877	0.3439
2	dog	0.2	1.6931	0.4521

In [43]:

```
from sklearn.metrics.pairwise import cosine_similarity

# Calculate cosine similarity using TF-IDF vectors
similarity_matrix = cosine_similarity(tfidf_matrix)

doc_names = ["D1", "D2", "D3"]

results = []

highest_similarity = -1
most_similar_pair = ""

for i in range(len(doc_names)):

    for j in range(i + 1, len(doc_names)):

        similarity = similarity_matrix[i][j]

        pair = doc_names[i] + " and " + doc_names[j]

        results.append({
            "Document Pair": pair,
            "Cosine Similarity": round(similarity, 4)
        })

        # Find the most similar pair
        if similarity > highest_similarity:
            highest_similarity = similarity
            most_similar_pair = pair

similarity_df = pd.DataFrame(results)

print("Cosine Similarity Results:")
display(similarity_df)

print("Most Similar Pair:", most_similar_pair)
print("Highest Similarity:", round(highest_similarity, 4))
```

```
Cosine Similarity Results:
```

	Document Pair	Cosine Similarity
0	D1 and D2	0.5387
1	D1 and D3	0.0000
2	D2 and D3	0.0000

```
Most Similar Pair: D1 and D2
Highest Similarity: 0.5387
```

Explanation of Results

d1 and d2 have the highest cosine similarity of 0 point 5387 because both documents contain the words the and cat d1 and d3 have zero cosine similarity because they do not share any exact words after preprocessing d2 and d3 also have zero cosine similarity because their words are different this matches my intuition because d1 and d2 both discuss a cat while d3 uses different words tfidf does not automatically recognize that cat and cats or dog and dogs have related meanings

D4

In [44]:

```
D1 = "The cat sat on the mat."
D2 = "The dog chased the cat."
D3 = "Dogs and cats are friendly pets."

documents = [D1, D2, D3]

def preprocess(text):

    # lowercase
    text = text.lower()

    # Remove punctuation
    text = text.translate(str.maketrans("", "", string.punctuation))

    # Split text into words
    return text.split()


# n-grams
def create_ngrams(words, n):

    result = []

    for i in range(len(words) - n + 1):
        result.append(" ".join(words[i:i+n]))

    return result


# Preprocess D1
words = preprocess(D1)

# Generate bigrams and trigrams
bigrams = create_ngrams(words, 2)
trigrams = create_ngrams(words, 3)

print("Bigrams of D1:")
for bigram in bigrams:
    print(bigram)

print("\nTrigrams of D1:")
for trigram in trigrams:
    print(trigram)
```

```
Bigrams of D1:
the cat
cat sat
sat on
on the
the mat

Trigrams of D1:
the cat sat
cat sat on
sat on the
on the mat
```

In [45]:

```
unique_unigrams = set()
unique_bigrams = set()

# Process each document separately
for document in documents:

    words = preprocess(document)

    # Add unique words
    unique_unigrams.update(words)

    # Generate and add unique bigrams
    bigrams = create_ngrams(words, 2)
    unique_bigrams.update(bigrams)

print("Unique Bigrams:")
for bigram in sorted(unique_bigrams):
    print(bigram)

print("\nUnigram Vocabulary Size:", len(unique_unigrams))
print("Unique Bigram Count:", len(unique_bigrams))
```

```
Unique Bigrams:
and cats
are friendly
cat sat
cats are
chased the
dog chased
dogs and
friendly pets
on the
sat on
the cat
the dog
the mat

Unigram Vocabulary Size: 13
Unique Bigram Count: 13
```

In [46]:

```python
from sklearn.feature_extraction.text import CountVectorizer

# Create CountVectorizer for unigrams and bigrams
vectorizer = CountVectorizer(ngram_range=(1, 2))

ngram_matrix = vectorizer.fit_transform(documents)

vocabulary = vectorizer.get_feature_names_out()


print("Unigrams and Bigrams:")
for word in vocabulary:
    print(word)

print("\nNew Vocabulary Size:", len(vocabulary))
```

```
Unigrams and Bigrams:
and
and cats
are
are friendly
cat
cat sat
cats
cats are
chased
chased the
dog
dog chased
dogs
dogs and
friendly
friendly pets
mat
on
on the
pets
sat
sat on
the
the cat
the dog
the mat

New Vocabulary Size: 26
```

Importance of Bigrams and Word Order

bigrams preserve some word order by combining two consecutive words for example not good and good not produce different bigrams even though they contain the same individual words the bigram not good helps preserve the negative meaning that a unigram representation may fail to capture

Cost of Larger N-grams

larger ngrams increase the vocabulary size and require more memory and processing time they also create sparse vectors because many word combinations occur rarely larger ngrams may not match similar sentences when their word sequences are different

Comparison Table:

Criteria	One-Hot	Bag of Words	TF-IDF
Captures word freque.	No	Yes	Yes
Capature word order.	No	No	No
Down-weights common word.	NO	NO	Yes

All three method fail in understanding Sentence such as distinugish between dog bites man and man bites dog."

E1

E1: CBOW and Skip-Gram by hand.

Sentence = "the quick brown fox jumps over the lazy dog"
Input = [quick, brown, jumps, over]
Output = fox ∵ CBOW predics words using its surrounding.

Skip-gram training pair.

input	output (context)
fox	quick
fox	brown
fox	jumps
fox	over

Total training pair = 4

Architecture for Rare Words?

Skip gram usually works better for rare words because it learns by predicting individual content words from the target word.

Nagative Sampling: Negative sampling is a training technique that teaches word2vec to distinugish actual target content pairs from randomly selected incorrect word pairs.

E2

In [47]:

```
%pip install -q gensim

from nltk.corpus import brown
from gensim.models import Word2Vec

# Download Brown corpus
nltk.download("brown", quiet=True)

# Load news category and convert all words to lowercase
sentences = [
    [word.lower() for word in sentence]
    for sentence in brown.sents(categories="news")
]

print("Total Sentences:", len(sentences))

# Train CBOW model
cbow_model = Word2Vec(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=5,
    sg=0,
    workers=1,
    seed=42
)

# Train Skip-Gram model
skipgram_model = Word2Vec(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=5,
    sg=1,
    workers=1,
    seed=42
)

print("\nBoth Word2Vec Models Trained Successfully")

# Display five nearest neighbours
words = ["money", "government"]

for word in words:

    print("\nWord:", word)

    for name, model in [
        ("CBOW", cbow_model),
        ("Skip-Gram", skipgram_model)
    ]:

        print("\n", name, "Nearest Neighbours:")

        if word in model.wv:
            neighbours = model.wv.most_similar(word, topn=5)

            for neighbour, score in neighbours:
                print(neighbour, ":", round(score, 4))
        else:
            print(word, "not found in vocabulary")


# Test analogy king - man + woman
print("\nAnalogy: king - man + woman")
```

```python
for name, model in [
    ("CBOW", cbow_model),
    ("Skip-Gram", skipgram_model)
]:

    print("\n", name, "Analogy Result:")

    required_words = ["king", "man", "woman"]

    if all(word in model.wv for word in required_words):

        result = model.wv.most_similar(
            positive=["king", "woman"],
            negative=["man"],
            topn=5
        )

        for word, score in result:
            print(word, ":", round(score, 4))

        if result[0][0] == "queen":
            print("Analogy Successful")
        else:
            print("Analogy did not return queen as the top result")

    else:
        print("Analogy cannot be completed")
        print("Some words are missing from the vocabulary")
```

```
[notice] A new release of pip is available: 24.0 -> 26.2.2
[notice] To update, run: D:\NLP-assignment1\nlp-env\Scripts\python.exe -m pip install --upgrade pip

Note: you may need to restart the kernel to use updated packages.
Total Sentences: 4623

Both Word2Vec Models Trained Successfully

Word: money

 CBOW Nearest Neighbours:
against : 0.9995
through : 0.9995
game : 0.9995
three : 0.9994
federal : 0.9994

 Skip-Gram Nearest Neighbours:
brought : 0.9965
proud : 0.9964
driven : 0.9963
working : 0.9963
reached : 0.9961

Word: government

 CBOW Nearest Neighbours:
being : 0.9997
against : 0.9997
up : 0.9996
an : 0.9996
now : 0.9996

 Skip-Gram Nearest Neighbours:
products : 0.9939
control : 0.9937
international : 0.9937
order : 0.9936
congo : 0.9935

Analogy: king - man + woman

 CBOW Analogy Result:
Analogy cannot be completed
Some words are missing from the vocabulary

 Skip-Gram Analogy Result:
Analogy cannot be completed
Some words are missing from the vocabulary
```

Comparison of CBOW and Skip Gram

both models were trained on the same brown news corpus with the same parameters except for their training architecture for the word money the cbow model returned [enter neighbours] while skip gram returned [enter neighbours] for the word government the cbow model returned [enter neighbours] while skip gram returned [enter neighbours] the results show [describe whether the models returned similar or different words] neither model can be considered better without evaluating the quality and relevance of its predictions

E3

In [48]:

```
import gensim.downloader as api

# Load pretrained GloVe model
glove = api.load("glove-wiki-gigaword-100")

# Part 1: Cosine similarity
pairs = {
    "synonym": ("car", "automobile"),
    "related": ("doctor", "hospital"),
    "unrelated": ("banana", "democracy")
}

for kind, (a_, b_) in pairs.items():
    print(f"{kind:<10} cos({a_}, {b_}) = {glove.similarity(a_, b_):.3f}")


# Part 2: Word analogies
def analogy(pos, neg, topn=3):
    return [
        (w, round(s, 3))
        for w, s in glove.most_similar(
            positive=pos,
            negative=neg,
            topn=topn
        )
    ]


# Analogy expected to work
works = analogy(["king", "woman"], ["man"])

print("\nAnalogy 1: king - man + woman")
print("Expected: queen")
print("Predictions:", works)
print("Successful:", works[0][0] == "queen")


# Analogy expected to fail
fails = analogy(["japan", "dollar"], ["yen"])

print("\nAnalogy 2: japan - yen + dollar")
print("Expected: usa or america")
print("Predictions:", fails)
print("Failed:", fails[0][0] not in ["usa", "america"])
```

```
[==================================================] 100.0% 128.1/128.1MB downloaded
synonym    cos(car, automobile) = 0.683
related    cos(doctor, hospital) = 0.690
unrelated  cos(banana, democracy) = 0.202

Analogy 1: king - man + woman
Expected: queen
Predictions: [('queen', 0.77), ('monarch', 0.684), ('throne', 0.676)]
Successful: True

Analogy 2: japan - yen + dollar
Expected: usa or america
Predictions: [('china', 0.667), ('asian', 0.664), ('world', 0.664)]
Failed: True
```

Explanation of Analogy Results

the first analogy uses king man and woman to predict queen by capturing the relationship between gender and royalty the second analogy uses japan yen and dollar to predict a country associated with the dollar this analogy may fail because the dollar is used by multiple countries and the relationship between currencies and countries is not always unique glove learns statistical relationships from text so it cannot always produce the expected answer

Difference Between GloVe and Word2Vec

glove learns word embeddings using global word cooccurrence statistics by minimizing a weighted error between vector dot products and logarithmic ooccurrence counts word2vec learns word embeddings by predicting a target word from its surrounding context or predicting surrounding words from a target word

E4

In [49]:
```
from gensim.models import Word2Vec, FastText
from nltk.corpus import brown

# Load Brown news corpus
sentences = [[w.lower() for w in s] for s in brown.sents(categories="news")]

# Train both models
w2v = Word2Vec(sentences, vector_size=100, window=5, min_count=5)
ft = FastText(sentences, vector_size=100, window=5, min_count=5)

# Select an OOV word
word = "governmentally"
if word in w2v.wv:
    word = "governmentallyxyz"

# FastText handles OOV words
print("FastText Vector:", ft.wv[word][:5])

# Word2Vec cannot handle OOV words
try:
    print("Word2Vec Vector:", w2v.wv[word][:5])
except KeyError:
    print("Word2Vec: Word not found in vocabulary")
```

```
FastText Vector: [-0.2210057   0.4925491  -0.24000265 -0.37168097  0.17954393]
Word2Vec: Word not found in vocabulary
```

Explanation of Results

fasttext can generate vectors for unseen words while word2vec cannot generate vectors for words outside its vocabulary fasttext divides words into smaller character ngrams and combines their learned representations to create a word vector this technique is especially useful for social media text containing misspelled words and unfamiliar word variations

E5

In [51]:

```
%pip install torch transformers

import torch
from transformers import AutoTokenizer, AutoModel
from torch.nn.functional import cosine_similarity


tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")
model.eval()

sentences = [
    "She sat on the river bank.",
    "He deposited the cash at the bank.",
    "The school deposited its savings at the bank."
]

def get_bank_vector(sentence):
    inputs = tokenizer(sentence, return_tensors="pt")
    tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])

    with torch.no_grad():
        output = model(**inputs).last_hidden_state

    return output[0, tokens.index("bank")]

vectors = [get_bank_vector(s) for s in sentences]

print("Sentence 1 vs 2:", round(cosine_similarity(vectors[0], vectors[1], dim=0).item(),
4))
print("Sentence 1 vs 3:", round(cosine_similarity(vectors[0], vectors[2], dim=0).item(),
4))
print("Sentence 2 vs 3:", round(cosine_similarity(vectors[1], vectors[2], dim=0).item(),
4))
```

```
[notice] A new release of pip is available: 24.0 -> 26.2.2
[notice] To update, run: D:\NLP-assignment1\nlp-env\Scripts\python.exe -m pip install --upgrade pip
```

```
Collecting torch
  Downloading torch-2.14.1-cp312-cp312-win_amd64.whl.metadata (38 kB)
Collecting transformers
  Downloading transformers-5.20.2-py3-none-any.whl.metadata (31 kB)
Collecting filelock (from torch)
  Downloading filelock-4.2.4-py3-none-any.whl.metadata (2.0 kB)
Requirement already satisfied: typing-extensions>=4.10.0 in d:\nlp-assignment1\nlp-env
\lib\site-packages (from torch) (4.16.0)
Requirement already satisfied: setuptools>=77.0.3 in d:\nlp-assignment1\nlp-env\lib\sit
e-packages (from torch) (84.2.0)
Collecting sympy>=1.13.3 (from torch)
  Using cached sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting networkx>=2.5.1 (from torch)
  Downloading networkx-3.7-py3-none-any.whl.metadata (6.7 kB)
Requirement already satisfied: jinja2 in d:\nlp-assignment1\nlp-env\lib\site-packages
(from torch) (3.1.6)
Collecting fsspec>=0.8.5 (from torch)
  Downloading fsspec-2026.9.0-py3-none-any.whl.metadata (10 kB)
Collecting huggingface-hub<3.0,>=1.31.0 (from transformers)
  Downloading huggingface_hub-2.2.0-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: numpy>=1.17 in d:\nlp-assignment1\nlp-env\lib\site-packa
ges (from transformers) (2.5.4)
Requirement already satisfied: packaging>=20.0 in d:\nlp-assignment1\nlp-env\lib\site-p
ackages (from transformers) (26.3)
Requirement already satisfied: pyyaml>=5.1 in d:\nlp-assignment1\nlp-env\lib\site-packa
ges (from transformers) (6.0.3)
Requirement already satisfied: regex>=2025.10.22 in d:\nlp-assignment1\nlp-env\lib\site
-packages (from transformers) (2026.9.29)
Collecting tokenizers<0.24.0,>=0.23.1 (from transformers)
  Downloading tokenizers-0.23.3-cp310-abi3-win_amd64.whl.metadata (10 kB)
Requirement already satisfied: typer in d:\nlp-assignment1\nlp-env\lib\site-packages (f
rom transformers) (0.27.3)
Collecting safetensors>=0.8.0 (from transformers)
  Downloading safetensors-0.8.0-cp310-abi3-win_amd64.whl.metadata (4.2 kB)
Requirement already satisfied: tqdm>=4.60 in d:\nlp-assignment1\nlp-env\lib\site-packag
es (from transformers) (4.70.1)
Requirement already satisfied: click<9.0.0,>=8.4.2 in d:\nlp-assignment1\nlp-env\lib\si
te-packages (from huggingface-hub<3.0,>=1.31.0->transformers) (8.5.0)
Collecting hf-xet<2.0.0,>=1.6.0 (from huggingface-hub<3.0,>=1.31.0->transformers)
  Downloading hf_xet-1.7.0-cp38-abi3-win_amd64.whl.metadata (4.9 kB)
Collecting httpx2<3,>=2.0.0 (from huggingface-hub<3.0,>=1.31.0->transformers)
  Downloading httpx2-2.16.0-py3-none-any.whl.metadata (10 kB)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.13.3->torch)
  Using cached mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: colorama in d:\nlp-assignment1\nlp-env\lib\site-packages
(from tqdm>=4.60->transformers) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in d:\nlp-assignment1\nlp-env\lib\site-p
ackages (from jinja2->torch) (3.0.4)
Requirement already satisfied: shellingham>=1.3.0 in d:\nlp-assignment1\nlp-env\lib\sit
e-packages (from typer->transformers) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in d:\nlp-assignment1\nlp-env\lib\site-pack
ages (from typer->transformers) (15.0.0)
Requirement already satisfied: annotated-doc>=0.0.2 in d:\nlp-assignment1\nlp-env\lib\s
ite-packages (from typer->transformers) (0.0.5)
Requirement already satisfied: anyio>=4.10 in d:\nlp-assignment1\nlp-env\lib\site-packa
ges (from httpx2<3,>=2.0.0->huggingface-hub<3.0,>=1.31.0->transformers) (4.15.1)
Collecting httpcore2==2.16.0 (from httpx2<3,>=2.0.0->huggingface-hub<3.0,>=1.31.0->tran
sformers)
  Downloading httpcore2-2.16.0-py3-none-any.whl.metadata (27 kB)
Requirement already satisfied: idna>=3.18 in d:\nlp-assignment1\nlp-env\lib\site-packag
es (from httpx2<3,>=2.0.0->huggingface-hub<3.0,>=1.31.0->transformers) (3.20)
Collecting truststore>=0.10 (from httpx2<3,>=2.0.0->huggingface-hub<3.0,>=1.31.0->trans
formers)
  Downloading truststore-0.10.5-py3-none-any.whl.metadata (4.5 kB)
Requirement already satisfied: h11>=0.16 in d:\nlp-assignment1\nlp-env\lib\site-package
s (from httpcore2==2.16.0->httpx2<3,>=2.0.0->huggingface-hub<3.0,>=1.31.0->transformer
s) (0.16.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in d:\nlp-assignment1\nlp-env\lib
\site-packages (from rich>=13.8.0->typer->transformers) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in d:\nlp-assignment1\nlp-env\li
b\site-packages (from rich>=13.8.0->typer->transformers) (2.21.0)
```

```
Requirement already satisfied: mdurl~=0.1 in d:\nlp-assignment1\nlp-env\lib\site-packag
es (from markdown-it-py>=2.2.0->rich>=13.8.0->typer->transformers) (0.1.2)
Downloading torch-2.14.1-cp312-cp312-win_amd64.whl (124.1 MB)
   ---------------------------------------- 0.0/124.1 MB ? eta -:--:--
   ---------------------------------------- 0.0/124.1 MB 1.9 MB/s eta 0:01:05
   ---------------------------------------- 0.0/124.1 MB 1.9 MB/s eta 0:01:05
   ---------------------------------------- 0.1/124.1 MB 744.7 kB/s eta 0:02:47
   ---------------------------------------- 0.1/124.1 MB 656.4 kB/s eta 0:03:09
   ---------------------------------------- 0.2/124.1 MB 784.3 kB/s eta 0:02:38
   ---------------------------------------- 0.2/124.1 MB 846.9 kB/s eta 0:02:27
   ---------------------------------------- 0.3/124.1 MB 863.3 kB/s eta 0:02:24
   ---------------------------------------- 0.5/124.1 MB 1.3 MB/s eta 0:01:39
   ---------------------------------------- 0.5/124.1 MB 1.2 MB/s eta 0:01:48
   ---------------------------------------- 0.9/124.1 MB 1.8 MB/s eta 0:01:09
   ---------------------------------------- 1.0/124.1 MB 1.9 MB/s eta 0:01:05
   ---------------------------------------- 1.2/124.1 MB 2.0 MB/s eta 0:01:02
   ---------------------------------------- 1.5/124.1 MB 2.3 MB/s eta 0:00:55
    --------------------------------------- 1.7/124.1 MB 2.4 MB/s eta 0:00:51
    --------------------------------------- 1.7/124.1 MB 2.3 MB/s eta 0:00:53
    --------------------------------------- 2.3/124.1 MB 3.0 MB/s eta 0:00:42
    --------------------------------------- 2.8/124.1 MB 3.3 MB/s eta 0:00:37
   - --------------------------------------- 3.1/124.1 MB 3.4 MB/s eta 0:00:36
   - --------------------------------------- 3.5/124.1 MB 3.6 MB/s eta 0:00:34
   - --------------------------------------- 3.9/124.1 MB 3.9 MB/s eta 0:00:32
   - --------------------------------------- 4.2/124.1 MB 4.0 MB/s eta 0:00:31
   - --------------------------------------- 4.4/124.1 MB 4.0 MB/s eta 0:00:30
   - --------------------------------------- 4.9/124.1 MB 4.2 MB/s eta 0:00:29
   - --------------------------------------- 5.3/124.1 MB 4.3 MB/s eta 0:00:28
   - --------------------------------------- 5.7/124.1 MB 4.4 MB/s eta 0:00:27
   - --------------------------------------- 5.9/124.1 MB 4.5 MB/s eta 0:00:27
   - --------------------------------------- 6.2/124.1 MB 4.5 MB/s eta 0:00:27
   -- -------------------------------------- 6.4/124.1 MB 4.4 MB/s eta 0:00:27
   -- -------------------------------------- 7.1/124.1 MB 4.7 MB/s eta 0:00:25
   -- -------------------------------------- 7.5/124.1 MB 4.8 MB/s eta 0:00:25
   -- -------------------------------------- 7.5/124.1 MB 4.8 MB/s eta 0:00:25
   -- -------------------------------------- 7.5/124.1 MB 4.8 MB/s eta 0:00:25
   -- -------------------------------------- 7.5/124.1 MB 4.8 MB/s eta 0:00:25
   -- -------------------------------------- 7.5/124.1 MB 4.8 MB/s eta 0:00:25
   -- -------------------------------------- 7.5/124.1 MB 4.8 MB/s eta 0:00:25
   -- -------------------------------------- 8.0/124.1 MB 4.0 MB/s eta 0:00:29
   -- -------------------------------------- 9.1/124.1 MB 4.5 MB/s eta 0:00:26
   --- ------------------------------------- 9.6/124.1 MB 4.7 MB/s eta 0:00:25
   --- ------------------------------------- 9.6/124.1 MB 4.7 MB/s eta 0:00:25
   --- ------------------------------------- 9.6/124.1 MB 4.7 MB/s eta 0:00:25
   --- ------------------------------------- 11.6/124.1 MB 6.2 MB/s eta 0:00:19
   ---- ------------------------------------ 12.7/124.1 MB 6.8 MB/s eta 0:00:17
   ---- ------------------------------------ 13.1/124.1 MB 6.8 MB/s eta 0:00:17
   ---- ------------------------------------ 13.4/124.1 MB 6.8 MB/s eta 0:00:17
   ---- ------------------------------------ 13.7/124.1 MB 6.6 MB/s eta 0:00:17
   ---- ------------------------------------ 13.8/124.1 MB 6.4 MB/s eta 0:00:18
   ---- ------------------------------------ 14.1/124.1 MB 6.4 MB/s eta 0:00:18
   ---- ------------------------------------ 14.4/124.1 MB 6.2 MB/s eta 0:00:18
   ---- ------------------------------------ 14.8/124.1 MB 6.3 MB/s eta 0:00:18
   ---- ------------------------------------ 15.2/124.1 MB 6.4 MB/s eta 0:00:18
   ---- ------------------------------------ 15.4/124.1 MB 6.3 MB/s eta 0:00:18
   ----- ----------------------------------- 15.7/124.1 MB 6.3 MB/s eta 0:00:18
   ----- ----------------------------------- 16.0/124.1 MB 6.3 MB/s eta 0:00:18
   ----- ----------------------------------- 16.6/124.1 MB 6.5 MB/s eta 0:00:17
   ----- ----------------------------------- 16.7/124.1 MB 6.5 MB/s eta 0:00:17
   ----- ----------------------------------- 16.9/124.1 MB 6.3 MB/s eta 0:00:18
   ----- ----------------------------------- 17.1/124.1 MB 6.1 MB/s eta 0:00:18
   ----- ----------------------------------- 17.3/124.1 MB 6.0 MB/s eta 0:00:18
   ----- ----------------------------------- 17.5/124.1 MB 5.8 MB/s eta 0:00:19
   ----- ----------------------------------- 17.9/124.1 MB 7.4 MB/s eta 0:00:15
   ----- ----------------------------------- 18.1/124.1 MB 7.2 MB/s eta 0:00:15
   ----- ----------------------------------- 18.1/124.1 MB 6.8 MB/s eta 0:00:16
   ------ ---------------------------------- 18.6/124.1 MB 6.6 MB/s eta 0:00:16
   ------ ---------------------------------- 18.9/124.1 MB 6.4 MB/s eta 0:00:17
   ------ ---------------------------------- 19.1/124.1 MB 6.4 MB/s eta 0:00:17
   ------ ---------------------------------- 19.1/124.1 MB 6.4 MB/s eta 0:00:17
   ------ ---------------------------------- 19.1/124.1 MB 6.4 MB/s eta 0:00:17
```

```
------ --------------------------------- 19.1/124.1 MB 6.4 MB/s eta 0:00:17
------ --------------------------------- 19.2/124.1 MB 5.5 MB/s eta 0:00:19
------ --------------------------------- 19.2/124.1 MB 5.5 MB/s eta 0:00:19
------ --------------------------------- 20.2/124.1 MB 5.7 MB/s eta 0:00:19
------ --------------------------------- 20.6/124.1 MB 5.5 MB/s eta 0:00:19
------ --------------------------------- 20.6/124.1 MB 5.5 MB/s eta 0:00:19
------ --------------------------------- 20.6/124.1 MB 5.5 MB/s eta 0:00:19
------ --------------------------------- 20.8/124.1 MB 5.1 MB/s eta 0:00:21
------ --------------------------------- 20.8/124.1 MB 5.1 MB/s eta 0:00:21
------ --------------------------------- 20.8/124.1 MB 5.1 MB/s eta 0:00:21
------ --------------------------------- 20.8/124.1 MB 5.1 MB/s eta 0:00:21
------ --------------------------------- 20.8/124.1 MB 4.5 MB/s eta 0:00:24
------- -------------------------------- 21.8/124.1 MB 4.5 MB/s eta 0:00:23
------- -------------------------------- 21.8/124.1 MB 4.5 MB/s eta 0:00:23
------- -------------------------------- 21.8/124.1 MB 4.5 MB/s eta 0:00:23
------- -------------------------------- 21.8/124.1 MB 4.5 MB/s eta 0:00:23
------- -------------------------------- 22.8/124.1 MB 4.1 MB/s eta 0:00:25
------- -------------------------------- 23.0/124.1 MB 4.1 MB/s eta 0:00:25
------- -------------------------------- 23.1/124.1 MB 4.1 MB/s eta 0:00:25
------- -------------------------------- 23.1/124.1 MB 4.1 MB/s eta 0:00:25
------- -------------------------------- 24.1/124.1 MB 4.2 MB/s eta 0:00:24
------- -------------------------------- 24.5/124.1 MB 4.2 MB/s eta 0:00:24
-------- ------------------------------- 25.2/124.1 MB 4.4 MB/s eta 0:00:23
-------- ------------------------------- 25.5/124.1 MB 4.4 MB/s eta 0:00:23
-------- ------------------------------- 25.5/124.1 MB 4.4 MB/s eta 0:00:23
-------- ------------------------------- 25.8/124.1 MB 4.3 MB/s eta 0:00:24
-------- ------------------------------- 25.9/124.1 MB 4.1 MB/s eta 0:00:24
-------- ------------------------------- 26.1/124.1 MB 4.2 MB/s eta 0:00:24
-------- ------------------------------- 26.2/124.1 MB 4.1 MB/s eta 0:00:24
-------- ------------------------------- 26.2/124.1 MB 4.1 MB/s eta 0:00:24
-------- ------------------------------- 26.9/124.1 MB 4.1 MB/s eta 0:00:24
-------- ------------------------------- 27.0/124.1 MB 4.1 MB/s eta 0:00:24
-------- ------------------------------- 27.1/124.1 MB 4.0 MB/s eta 0:00:25
-------- ------------------------------- 27.2/124.1 MB 3.9 MB/s eta 0:00:25
-------- ------------------------------- 27.4/124.1 MB 3.9 MB/s eta 0:00:25
-------- ------------------------------- 27.5/124.1 MB 3.8 MB/s eta 0:00:26
-------- ------------------------------- 27.6/124.1 MB 3.8 MB/s eta 0:00:26
-------- ------------------------------- 27.7/124.1 MB 3.8 MB/s eta 0:00:26
-------- ------------------------------- 27.8/124.1 MB 3.7 MB/s eta 0:00:26
-------- ------------------------------- 27.9/124.1 MB 3.7 MB/s eta 0:00:27
--------- ------------------------------ 28.0/124.1 MB 3.7 MB/s eta 0:00:27
--------- ------------------------------ 28.1/124.1 MB 3.6 MB/s eta 0:00:27
--------- ------------------------------ 28.2/124.1 MB 3.6 MB/s eta 0:00:27
--------- ------------------------------ 28.3/124.1 MB 3.6 MB/s eta 0:00:27
--------- ------------------------------ 28.4/124.1 MB 3.6 MB/s eta 0:00:27
--------- ------------------------------ 28.5/124.1 MB 3.6 MB/s eta 0:00:27
--------- ------------------------------ 28.6/124.1 MB 3.5 MB/s eta 0:00:27
--------- ------------------------------ 28.7/124.1 MB 3.5 MB/s eta 0:00:28
--------- ------------------------------ 28.9/124.1 MB 3.4 MB/s eta 0:00:28
--------- ------------------------------ 28.9/124.1 MB 3.4 MB/s eta 0:00:29
--------- ------------------------------ 29.0/124.1 MB 3.4 MB/s eta 0:00:29
--------- ------------------------------ 29.2/124.1 MB 3.3 MB/s eta 0:00:29
--------- ------------------------------ 29.2/124.1 MB 3.3 MB/s eta 0:00:29
--------- ------------------------------ 29.5/124.1 MB 3.7 MB/s eta 0:00:26
--------- ------------------------------ 29.6/124.1 MB 3.6 MB/s eta 0:00:27
--------- ------------------------------ 29.7/124.1 MB 3.5 MB/s eta 0:00:27
--------- ------------------------------ 29.7/124.1 MB 3.5 MB/s eta 0:00:28
--------- ------------------------------ 30.0/124.1 MB 3.4 MB/s eta 0:00:28
--------- ------------------------------ 30.1/124.1 MB 3.3 MB/s eta 0:00:29
--------- ------------------------------ 30.2/124.1 MB 3.3 MB/s eta 0:00:29
--------- ------------------------------ 30.3/124.1 MB 3.2 MB/s eta 0:00:30
--------- ------------------------------ 30.6/124.1 MB 3.2 MB/s eta 0:00:30
--------- ------------------------------ 30.6/124.1 MB 3.1 MB/s eta 0:00:30
--------- ------------------------------ 30.8/124.1 MB 3.1 MB/s eta 0:00:31
--------- ------------------------------ 30.8/124.1 MB 3.0 MB/s eta 0:00:31
--------- ------------------------------ 31.0/124.1 MB 3.1 MB/s eta 0:00:30
---------- ----------------------------- 31.1/124.1 MB 3.4 MB/s eta 0:00:28
---------- ----------------------------- 31.2/124.1 MB 3.3 MB/s eta 0:00:28
---------- ----------------------------- 31.4/124.1 MB 3.3 MB/s eta 0:00:29
---------- ----------------------------- 31.5/124.1 MB 3.2 MB/s eta 0:00:29
---------- ----------------------------- 31.6/124.1 MB 3.2 MB/s eta 0:00:30
```

---------- ---------------------------- 31.7/124.1 MB 3.1 MB/s eta 0:00:30
---------- ---------------------------- 31.8/124.1 MB 3.1 MB/s eta 0:00:30
---------- ---------------------------- 31.9/124.1 MB 3.0 MB/s eta 0:00:31
---------- ---------------------------- 32.1/124.1 MB 3.0 MB/s eta 0:00:31
---------- ---------------------------- 32.2/124.1 MB 3.1 MB/s eta 0:00:30
---------- ---------------------------- 32.3/124.1 MB 3.1 MB/s eta 0:00:30
---------- ---------------------------- 32.4/124.1 MB 3.0 MB/s eta 0:00:31
---------- ---------------------------- 32.5/124.1 MB 3.0 MB/s eta 0:00:31
---------- ---------------------------- 32.6/124.1 MB 3.0 MB/s eta 0:00:31
---------- ---------------------------- 32.7/124.1 MB 2.9 MB/s eta 0:00:32
---------- ---------------------------- 32.8/124.1 MB 2.9 MB/s eta 0:00:32
---------- ---------------------------- 33.0/124.1 MB 2.8 MB/s eta 0:00:33
---------- ---------------------------- 33.0/124.1 MB 2.8 MB/s eta 0:00:33
---------- ---------------------------- 33.1/124.1 MB 2.8 MB/s eta 0:00:33
---------- ---------------------------- 33.3/124.1 MB 2.8 MB/s eta 0:00:33
---------- ---------------------------- 33.4/124.1 MB 2.8 MB/s eta 0:00:32
---------- ---------------------------- 33.5/124.1 MB 2.8 MB/s eta 0:00:33
---------- ---------------------------- 33.6/124.1 MB 2.8 MB/s eta 0:00:33
---------- ---------------------------- 33.8/124.1 MB 2.8 MB/s eta 0:00:33
---------- ---------------------------- 33.9/124.1 MB 2.7 MB/s eta 0:00:34
---------- ---------------------------- 34.0/124.1 MB 2.7 MB/s eta 0:00:34
---------- ---------------------------- 34.1/124.1 MB 2.7 MB/s eta 0:00:34
----------- ---------------------------- 34.2/124.1 MB 2.6 MB/s eta 0:00:35
----------- ---------------------------- 34.4/124.1 MB 2.6 MB/s eta 0:00:35
----------- ---------------------------- 34.4/124.1 MB 2.6 MB/s eta 0:00:36
----------- ---------------------------- 34.6/124.1 MB 2.5 MB/s eta 0:00:36
----------- ---------------------------- 34.7/124.1 MB 2.5 MB/s eta 0:00:36
----------- ---------------------------- 34.8/124.1 MB 2.5 MB/s eta 0:00:37
----------- ---------------------------- 34.9/124.1 MB 2.5 MB/s eta 0:00:37
----------- ---------------------------- 35.0/124.1 MB 2.4 MB/s eta 0:00:37
----------- ---------------------------- 35.2/124.1 MB 2.4 MB/s eta 0:00:38
----------- ---------------------------- 35.3/124.1 MB 2.4 MB/s eta 0:00:38
----------- ---------------------------- 35.5/124.1 MB 2.3 MB/s eta 0:00:38
----------- ---------------------------- 35.6/124.1 MB 2.3 MB/s eta 0:00:38
----------- ---------------------------- 35.7/124.1 MB 2.3 MB/s eta 0:00:38
----------- ---------------------------- 35.8/124.1 MB 2.4 MB/s eta 0:00:38
----------- ---------------------------- 36.0/124.1 MB 2.3 MB/s eta 0:00:38
----------- ---------------------------- 36.0/124.1 MB 2.3 MB/s eta 0:00:39
----------- ---------------------------- 36.1/124.1 MB 2.3 MB/s eta 0:00:39
----------- ---------------------------- 36.3/124.1 MB 2.3 MB/s eta 0:00:39
----------- ---------------------------- 36.4/124.1 MB 2.3 MB/s eta 0:00:39
----------- ---------------------------- 36.4/124.1 MB 2.3 MB/s eta 0:00:39
----------- ---------------------------- 36.6/124.1 MB 2.3 MB/s eta 0:00:38
----------- ---------------------------- 36.7/124.1 MB 2.3 MB/s eta 0:00:39
----------- ---------------------------- 36.7/124.1 MB 2.3 MB/s eta 0:00:39
----------- ---------------------------- 36.9/124.1 MB 2.2 MB/s eta 0:00:40
----------- ---------------------------- 37.0/124.1 MB 2.2 MB/s eta 0:00:40
----------- ---------------------------- 37.1/124.1 MB 2.2 MB/s eta 0:00:40
----------- ---------------------------- 37.2/124.1 MB 2.2 MB/s eta 0:00:40
------------ ---------------------------- 37.3/124.1 MB 2.2 MB/s eta 0:00:40
------------ ---------------------------- 37.5/124.1 MB 2.2 MB/s eta 0:00:39
------------ ---------------------------- 37.6/124.1 MB 2.2 MB/s eta 0:00:39
------------ ---------------------------- 37.6/124.1 MB 2.2 MB/s eta 0:00:39
------------ ---------------------------- 37.7/124.1 MB 2.2 MB/s eta 0:00:40
------------ ---------------------------- 37.9/124.1 MB 2.2 MB/s eta 0:00:40
------------ ---------------------------- 38.0/124.1 MB 2.2 MB/s eta 0:00:40
------------ ---------------------------- 38.1/124.1 MB 2.2 MB/s eta 0:00:40
------------ ---------------------------- 38.2/124.1 MB 2.2 MB/s eta 0:00:39
------------ ---------------------------- 38.4/124.1 MB 2.2 MB/s eta 0:00:39
------------ ---------------------------- 38.5/124.1 MB 2.2 MB/s eta 0:00:39
------------ ---------------------------- 38.6/124.1 MB 2.2 MB/s eta 0:00:39
------------ ---------------------------- 38.7/124.1 MB 2.2 MB/s eta 0:00:39
------------ ---------------------------- 38.8/124.1 MB 2.2 MB/s eta 0:00:39
------------ ---------------------------- 39.0/124.1 MB 2.2 MB/s eta 0:00:38
------------ ---------------------------- 39.1/124.1 MB 2.2 MB/s eta 0:00:38
------------ ---------------------------- 39.2/124.1 MB 2.2 MB/s eta 0:00:38
------------ ---------------------------- 39.4/124.1 MB 2.2 MB/s eta 0:00:38
------------ ---------------------------- 39.5/124.1 MB 2.2 MB/s eta 0:00:38
------------ ---------------------------- 39.6/124.1 MB 2.3 MB/s eta 0:00:38
------------ ---------------------------- 39.6/124.1 MB 2.2 MB/s eta 0:00:38
------------ ---------------------------- 39.8/124.1 MB 2.2 MB/s eta 0:00:38

```
------------ --------------------------- 40.0/124.1 MB 2.3 MB/s eta 0:00:37
------------ --------------------------- 40.0/124.1 MB 2.3 MB/s eta 0:00:37
------------ --------------------------- 40.1/124.1 MB 2.3 MB/s eta 0:00:38
------------ --------------------------- 40.2/124.1 MB 2.2 MB/s eta 0:00:38
------------ --------------------------- 40.3/124.1 MB 2.2 MB/s eta 0:00:38
------------ --------------------------- 40.5/124.1 MB 2.2 MB/s eta 0:00:38
------------ --------------------------- 40.7/124.1 MB 2.2 MB/s eta 0:00:38
------------ --------------------------- 40.8/124.1 MB 2.2 MB/s eta 0:00:38
------------ --------------------------- 40.9/124.1 MB 2.2 MB/s eta 0:00:38
------------ --------------------------- 41.1/124.1 MB 2.3 MB/s eta 0:00:37
------------ --------------------------- 41.1/124.1 MB 2.2 MB/s eta 0:00:38
------------ --------------------------- 41.3/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 41.4/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 41.5/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 41.6/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 41.7/124.1 MB 2.3 MB/s eta 0:00:37
------------ --------------------------- 41.8/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 41.9/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 42.1/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 42.1/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 42.3/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 42.4/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 42.5/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 42.6/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 42.8/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 42.9/124.1 MB 2.3 MB/s eta 0:00:36
------------ --------------------------- 42.9/124.1 MB 2.3 MB/s eta 0:00:36
------------ --------------------------- 42.9/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 43.0/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 43.2/124.1 MB 2.2 MB/s eta 0:00:37
------------ --------------------------- 43.4/124.1 MB 2.2 MB/s eta 0:00:37
------------- -------------------------- 43.5/124.1 MB 2.2 MB/s eta 0:00:37
------------- -------------------------- 43.6/124.1 MB 2.2 MB/s eta 0:00:37
------------- -------------------------- 43.7/124.1 MB 2.2 MB/s eta 0:00:37
------------- -------------------------- 43.9/124.1 MB 2.2 MB/s eta 0:00:37
------------- -------------------------- 44.0/124.1 MB 2.2 MB/s eta 0:00:37
------------- -------------------------- 44.1/124.1 MB 2.2 MB/s eta 0:00:37
------------- -------------------------- 44.2/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 44.4/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 44.5/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 44.7/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 44.8/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 44.9/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 45.1/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 45.2/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 45.4/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 45.5/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 45.6/124.1 MB 2.2 MB/s eta 0:00:36
------------- -------------------------- 45.9/124.1 MB 2.2 MB/s eta 0:00:35
------------- -------------------------- 46.0/124.1 MB 2.3 MB/s eta 0:00:35
------------- -------------------------- 46.2/124.1 MB 2.3 MB/s eta 0:00:35
------------- -------------------------- 46.2/124.1 MB 2.3 MB/s eta 0:00:35
------------- -------------------------- 46.4/124.1 MB 2.3 MB/s eta 0:00:35
------------- -------------------------- 46.5/124.1 MB 2.3 MB/s eta 0:00:35
-------------- ------------------------- 46.6/124.1 MB 2.3 MB/s eta 0:00:34
-------------- ------------------------- 46.7/124.1 MB 2.3 MB/s eta 0:00:34
-------------- ------------------------- 46.8/124.1 MB 2.3 MB/s eta 0:00:35
-------------- ------------------------- 46.9/124.1 MB 2.3 MB/s eta 0:00:34
-------------- ------------------------- 47.1/124.1 MB 2.3 MB/s eta 0:00:34
-------------- ------------------------- 47.2/124.1 MB 2.3 MB/s eta 0:00:34
-------------- ------------------------- 47.3/124.1 MB 2.3 MB/s eta 0:00:34
-------------- ------------------------- 47.5/124.1 MB 2.3 MB/s eta 0:00:34
-------------- ------------------------- 47.7/124.1 MB 2.3 MB/s eta 0:00:33
-------------- ------------------------- 47.8/124.1 MB 2.3 MB/s eta 0:00:33
-------------- ------------------------- 47.9/124.1 MB 2.4 MB/s eta 0:00:33
-------------- ------------------------- 48.1/124.1 MB 2.4 MB/s eta 0:00:33
-------------- ------------------------- 48.2/124.1 MB 2.4 MB/s eta 0:00:33
-------------- ------------------------- 48.3/124.1 MB 2.4 MB/s eta 0:00:32
-------------- ------------------------- 48.5/124.1 MB 2.4 MB/s eta 0:00:32
-------------- ------------------------- 48.6/124.1 MB 2.4 MB/s eta 0:00:32
-------------- ------------------------- 48.8/124.1 MB 2.4 MB/s eta 0:00:32
```

```
-------------- ---------------------- 49.0/124.1 MB 2.4 MB/s eta 0:00:32
-------------- ---------------------- 49.1/124.1 MB 2.4 MB/s eta 0:00:32
-------------- ---------------------- 49.3/124.1 MB 2.4 MB/s eta 0:00:31
-------------- ---------------------- 49.4/124.1 MB 2.4 MB/s eta 0:00:31
-------------- ---------------------- 49.5/124.1 MB 2.4 MB/s eta 0:00:31
--------------- ---------------------- 49.7/124.1 MB 2.4 MB/s eta 0:00:31
--------------- ---------------------- 49.8/124.1 MB 2.4 MB/s eta 0:00:31
--------------- ---------------------- 50.0/124.1 MB 2.5 MB/s eta 0:00:31
--------------- ---------------------- 50.1/124.1 MB 2.4 MB/s eta 0:00:31
--------------- ---------------------- 50.3/124.1 MB 2.5 MB/s eta 0:00:30
--------------- ---------------------- 50.4/124.1 MB 2.5 MB/s eta 0:00:30
--------------- ---------------------- 50.5/124.1 MB 2.5 MB/s eta 0:00:30
--------------- ---------------------- 50.7/124.1 MB 2.5 MB/s eta 0:00:30
--------------- ---------------------- 50.8/124.1 MB 2.5 MB/s eta 0:00:30
--------------- ---------------------- 50.9/124.1 MB 2.5 MB/s eta 0:00:30
--------------- ---------------------- 51.1/124.1 MB 2.5 MB/s eta 0:00:29
--------------- ---------------------- 51.2/124.1 MB 2.5 MB/s eta 0:00:29
--------------- ---------------------- 51.5/124.1 MB 2.5 MB/s eta 0:00:29
--------------- ---------------------- 51.7/124.1 MB 2.6 MB/s eta 0:00:29
--------------- ---------------------- 51.8/124.1 MB 2.6 MB/s eta 0:00:29
--------------- ---------------------- 51.9/124.1 MB 2.6 MB/s eta 0:00:29
--------------- ---------------------- 52.1/124.1 MB 2.6 MB/s eta 0:00:28
--------------- ---------------------- 52.3/124.1 MB 2.6 MB/s eta 0:00:28
--------------- ---------------------- 52.5/124.1 MB 2.7 MB/s eta 0:00:28
--------------- ---------------------- 52.7/124.1 MB 2.6 MB/s eta 0:00:28
---------------- ---------------------- 52.9/124.1 MB 2.7 MB/s eta 0:00:27
---------------- ---------------------- 53.0/124.1 MB 2.7 MB/s eta 0:00:27
---------------- ---------------------- 53.3/124.1 MB 2.8 MB/s eta 0:00:26
---------------- ---------------------- 53.5/124.1 MB 2.8 MB/s eta 0:00:26
---------------- ---------------------- 53.7/124.1 MB 2.8 MB/s eta 0:00:26
---------------- ---------------------- 53.9/124.1 MB 2.8 MB/s eta 0:00:25
---------------- ---------------------- 54.0/124.1 MB 2.8 MB/s eta 0:00:25
---------------- ---------------------- 54.3/124.1 MB 2.9 MB/s eta 0:00:25
---------------- ---------------------- 54.4/124.1 MB 2.9 MB/s eta 0:00:25
---------------- ---------------------- 54.6/124.1 MB 2.9 MB/s eta 0:00:25
---------------- ---------------------- 54.8/124.1 MB 2.9 MB/s eta 0:00:24
---------------- ---------------------- 54.9/124.1 MB 2.9 MB/s eta 0:00:24
---------------- ---------------------- 55.1/124.1 MB 3.0 MB/s eta 0:00:24
---------------- ---------------------- 55.3/124.1 MB 3.0 MB/s eta 0:00:24
---------------- ---------------------- 55.4/124.1 MB 3.0 MB/s eta 0:00:24
---------------- ---------------------- 55.7/124.1 MB 3.0 MB/s eta 0:00:23
---------------- ---------------------- 55.8/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 56.0/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 56.0/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 56.1/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 56.4/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 56.5/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 56.5/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 56.5/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 56.5/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 56.6/124.1 MB 2.9 MB/s eta 0:00:24
----------------- ---------------------- 56.9/124.1 MB 2.9 MB/s eta 0:00:24
----------------- ---------------------- 57.0/124.1 MB 2.9 MB/s eta 0:00:23
----------------- ---------------------- 57.4/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 57.5/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 57.7/124.1 MB 3.0 MB/s eta 0:00:23
----------------- ---------------------- 58.0/124.1 MB 3.1 MB/s eta 0:00:22
----------------- ---------------------- 58.2/124.1 MB 3.1 MB/s eta 0:00:22
----------------- ---------------------- 58.4/124.1 MB 3.1 MB/s eta 0:00:22
----------------- ---------------------- 58.7/124.1 MB 3.1 MB/s eta 0:00:22
----------------- ---------------------- 58.9/124.1 MB 3.1 MB/s eta 0:00:21
------------------ --------------------- 59.2/124.1 MB 3.2 MB/s eta 0:00:21
------------------ --------------------- 59.5/124.1 MB 3.2 MB/s eta 0:00:21
------------------ --------------------- 59.7/124.1 MB 3.2 MB/s eta 0:00:20
------------------ --------------------- 59.9/124.1 MB 3.3 MB/s eta 0:00:20
------------------ --------------------- 60.2/124.1 MB 3.3 MB/s eta 0:00:20
------------------ --------------------- 60.5/124.1 MB 3.4 MB/s eta 0:00:19
------------------ --------------------- 60.9/124.1 MB 3.4 MB/s eta 0:00:19
------------------ --------------------- 61.0/124.1 MB 3.4 MB/s eta 0:00:19
------------------ --------------------- 61.1/124.1 MB 3.5 MB/s eta 0:00:18
------------------ --------------------- 61.2/124.1 MB 3.4 MB/s eta 0:00:19
```

```
------------------ ------------------- 61.6/124.1 MB 3.5 MB/s eta 0:00:18
------------------ ------------------- 61.8/124.1 MB 3.5 MB/s eta 0:00:18
------------------ ------------------- 61.9/124.1 MB 3.6 MB/s eta 0:00:18
------------------- ------------------ 62.1/124.1 MB 3.5 MB/s eta 0:00:18
------------------- ------------------ 62.4/124.1 MB 3.6 MB/s eta 0:00:18
------------------- ------------------ 62.7/124.1 MB 3.6 MB/s eta 0:00:17
------------------- ------------------ 63.0/124.1 MB 3.7 MB/s eta 0:00:17
------------------- ------------------ 63.3/124.1 MB 3.7 MB/s eta 0:00:17
------------------- ------------------ 63.6/124.1 MB 3.7 MB/s eta 0:00:17
------------------- ------------------ 63.7/124.1 MB 3.7 MB/s eta 0:00:17
------------------- ------------------ 63.8/124.1 MB 3.6 MB/s eta 0:00:17
------------------- ------------------ 64.0/124.1 MB 3.7 MB/s eta 0:00:17
------------------- ------------------ 64.4/124.1 MB 3.7 MB/s eta 0:00:17
------------------- ------------------ 64.6/124.1 MB 3.7 MB/s eta 0:00:17
------------------- ------------------ 64.8/124.1 MB 3.8 MB/s eta 0:00:16
------------------- ------------------ 65.1/124.1 MB 3.9 MB/s eta 0:00:16
-------------------- ------------------ 65.3/124.1 MB 3.8 MB/s eta 0:00:16
-------------------- ------------------ 65.6/124.1 MB 3.9 MB/s eta 0:00:16
-------------------- ------------------ 65.9/124.1 MB 3.9 MB/s eta 0:00:15
-------------------- ------------------ 66.1/124.1 MB 3.9 MB/s eta 0:00:15
-------------------- ------------------ 66.5/124.1 MB 4.1 MB/s eta 0:00:15
-------------------- ------------------ 66.9/124.1 MB 4.5 MB/s eta 0:00:13
-------------------- ------------------ 67.1/124.1 MB 4.5 MB/s eta 0:00:13
-------------------- ------------------ 67.4/124.1 MB 4.7 MB/s eta 0:00:13
-------------------- ------------------ 67.8/124.1 MB 4.7 MB/s eta 0:00:12
-------------------- ------------------ 68.1/124.1 MB 4.8 MB/s eta 0:00:12
--------------------- ------------------ 68.5/124.1 MB 4.9 MB/s eta 0:00:12
--------------------- ------------------ 68.7/124.1 MB 4.9 MB/s eta 0:00:12
--------------------- ------------------ 69.1/124.1 MB 4.9 MB/s eta 0:00:12
--------------------- ------------------ 69.3/124.1 MB 5.0 MB/s eta 0:00:12
--------------------- ------------------ 69.7/124.1 MB 5.0 MB/s eta 0:00:11
--------------------- ------------------ 70.1/124.1 MB 5.1 MB/s eta 0:00:11
--------------------- ------------------ 70.5/124.1 MB 5.2 MB/s eta 0:00:11
--------------------- ------------------ 70.8/124.1 MB 5.2 MB/s eta 0:00:11
--------------------- ------------------ 71.1/124.1 MB 5.2 MB/s eta 0:00:11
---------------------- ----------------- 71.6/124.1 MB 5.5 MB/s eta 0:00:10
---------------------- ----------------- 71.8/124.1 MB 5.4 MB/s eta 0:00:10
---------------------- ----------------- 72.3/124.1 MB 5.6 MB/s eta 0:00:10
---------------------- ----------------- 72.7/124.1 MB 5.7 MB/s eta 0:00:09
---------------------- ----------------- 73.0/124.1 MB 5.7 MB/s eta 0:00:09
---------------------- ----------------- 73.4/124.1 MB 5.8 MB/s eta 0:00:09
---------------------- ----------------- 73.6/124.1 MB 5.8 MB/s eta 0:00:09
---------------------- ----------------- 74.2/124.1 MB 6.2 MB/s eta 0:00:09
----------------------- ---------------- 74.5/124.1 MB 6.2 MB/s eta 0:00:08
----------------------- ---------------- 74.8/124.1 MB 6.3 MB/s eta 0:00:08
----------------------- ---------------- 74.9/124.1 MB 6.2 MB/s eta 0:00:08
----------------------- ---------------- 74.9/124.1 MB 6.2 MB/s eta 0:00:08
----------------------- ---------------- 75.1/124.1 MB 6.0 MB/s eta 0:00:09
----------------------- ---------------- 75.9/124.1 MB 6.4 MB/s eta 0:00:08
----------------------- ---------------- 76.3/124.1 MB 6.5 MB/s eta 0:00:08
----------------------- ---------------- 76.3/124.1 MB 6.5 MB/s eta 0:00:08
----------------------- ---------------- 76.3/124.1 MB 6.5 MB/s eta 0:00:08
----------------------- ---------------- 76.6/124.1 MB 6.2 MB/s eta 0:00:08
----------------------- ---------------- 76.8/124.1 MB 6.2 MB/s eta 0:00:08
----------------------- ---------------- 77.0/124.1 MB 6.1 MB/s eta 0:00:08
----------------------- ---------------- 77.4/124.1 MB 6.1 MB/s eta 0:00:08
------------------------ --------------- 77.9/124.1 MB 6.1 MB/s eta 0:00:08
------------------------ --------------- 78.1/124.1 MB 6.0 MB/s eta 0:00:08
------------------------ --------------- 78.2/124.1 MB 6.0 MB/s eta 0:00:08
------------------------ --------------- 78.3/124.1 MB 5.9 MB/s eta 0:00:08
------------------------ --------------- 78.5/124.1 MB 5.7 MB/s eta 0:00:08
------------------------ --------------- 78.8/124.1 MB 5.6 MB/s eta 0:00:09
------------------------ --------------- 79.0/124.1 MB 5.6 MB/s eta 0:00:09
------------------------ --------------- 79.0/124.1 MB 5.6 MB/s eta 0:00:09
------------------------ --------------- 79.0/124.1 MB 5.6 MB/s eta 0:00:09
------------------------ --------------- 79.0/124.1 MB 5.6 MB/s eta 0:00:09
------------------------ --------------- 79.3/124.1 MB 5.1 MB/s eta 0:00:09
------------------------ --------------- 79.4/124.1 MB 5.1 MB/s eta 0:00:09
------------------------ --------------- 79.4/124.1 MB 5.1 MB/s eta 0:00:09
------------------------ --------------- 79.6/124.1 MB 4.9 MB/s eta 0:00:10
------------------------ --------------- 79.8/124.1 MB 4.8 MB/s eta 0:00:10
```

```
---------------------- -------------- 80.0/124.1 MB 4.8 MB/s eta 0:00:10
---------------------- -------------- 80.1/124.1 MB 4.7 MB/s eta 0:00:10
---------------------- -------------- 80.2/124.1 MB 4.6 MB/s eta 0:00:10
---------------------- -------------- 80.3/124.1 MB 4.5 MB/s eta 0:00:10
---------------------- -------------- 80.5/124.1 MB 4.5 MB/s eta 0:00:10
---------------------- -------------- 80.6/124.1 MB 4.5 MB/s eta 0:00:10
---------------------- -------------- 80.8/124.1 MB 4.4 MB/s eta 0:00:10
---------------------- -------------- 81.0/124.1 MB 4.3 MB/s eta 0:00:10
---------------------- -------------- 81.1/124.1 MB 4.3 MB/s eta 0:00:11
---------------------- -------------- 81.3/124.1 MB 4.2 MB/s eta 0:00:11
---------------------- -------------- 81.4/124.1 MB 4.2 MB/s eta 0:00:11
---------------------- -------------- 81.5/124.1 MB 4.2 MB/s eta 0:00:11
---------------------- -------------- 81.6/124.1 MB 4.1 MB/s eta 0:00:11
---------------------- -------------- 81.7/124.1 MB 4.0 MB/s eta 0:00:11
---------------------- -------------- 82.0/124.1 MB 3.9 MB/s eta 0:00:11
---------------------- -------------- 82.2/124.1 MB 3.9 MB/s eta 0:00:11
---------------------- -------------- 82.2/124.1 MB 3.9 MB/s eta 0:00:11
---------------------- -------------- 82.2/124.1 MB 3.8 MB/s eta 0:00:12
---------------------- -------------- 82.5/124.1 MB 3.7 MB/s eta 0:00:12
---------------------- -------------- 82.6/124.1 MB 3.7 MB/s eta 0:00:12
---------------------- -------------- 82.8/124.1 MB 3.7 MB/s eta 0:00:12
---------------------- -------------- 82.9/124.1 MB 3.7 MB/s eta 0:00:12
---------------------- -------------- 83.0/124.1 MB 3.6 MB/s eta 0:00:12
---------------------- -------------- 83.3/124.1 MB 3.6 MB/s eta 0:00:12
---------------------- -------------- 83.5/124.1 MB 3.5 MB/s eta 0:00:12
---------------------- -------------- 83.6/124.1 MB 3.5 MB/s eta 0:00:12
---------------------- ------------- 83.8/124.1 MB 3.5 MB/s eta 0:00:12
---------------------- ------------- 83.9/124.1 MB 3.5 MB/s eta 0:00:12
---------------------- ------------- 84.2/124.1 MB 3.4 MB/s eta 0:00:12
---------------------- ------------- 84.3/124.1 MB 3.4 MB/s eta 0:00:12
---------------------- ------------- 84.4/124.1 MB 3.3 MB/s eta 0:00:12
---------------------- ------------- 84.5/124.1 MB 3.3 MB/s eta 0:00:13
---------------------- ------------- 84.7/124.1 MB 3.3 MB/s eta 0:00:13
---------------------- ------------- 84.8/124.1 MB 3.2 MB/s eta 0:00:13
---------------------- ------------- 84.9/124.1 MB 3.2 MB/s eta 0:00:13
---------------------- ------------- 85.0/124.1 MB 3.1 MB/s eta 0:00:13
---------------------- ------------- 85.2/124.1 MB 3.2 MB/s eta 0:00:13
---------------------- ------------- 85.3/124.1 MB 3.2 MB/s eta 0:00:13
---------------------- ------------- 85.4/124.1 MB 3.2 MB/s eta 0:00:13
---------------------- ------------- 85.5/124.1 MB 3.1 MB/s eta 0:00:13
---------------------- ------------- 85.6/124.1 MB 3.1 MB/s eta 0:00:13
---------------------- ------------- 85.7/124.1 MB 3.0 MB/s eta 0:00:13
---------------------- ------------- 85.8/124.1 MB 3.0 MB/s eta 0:00:13
---------------------- ------------- 85.9/124.1 MB 2.9 MB/s eta 0:00:13
---------------------- ------------- 86.0/124.1 MB 2.9 MB/s eta 0:00:14
---------------------- ------------- 86.1/124.1 MB 2.9 MB/s eta 0:00:14
---------------------- ------------- 86.2/124.1 MB 2.9 MB/s eta 0:00:14
---------------------- ------------- 86.3/124.1 MB 2.8 MB/s eta 0:00:14
---------------------- ------------- 86.6/124.1 MB 2.9 MB/s eta 0:00:14
---------------------- ------------- 86.7/124.1 MB 2.8 MB/s eta 0:00:14
---------------------- ------------- 86.8/124.1 MB 2.8 MB/s eta 0:00:14
---------------------- ------------ 87.1/124.1 MB 2.8 MB/s eta 0:00:14
---------------------- ------------ 87.2/124.1 MB 2.8 MB/s eta 0:00:13
---------------------- ------------ 87.3/124.1 MB 2.8 MB/s eta 0:00:14
---------------------- ------------ 87.4/124.1 MB 2.8 MB/s eta 0:00:14
---------------------- ------------ 87.6/124.1 MB 2.8 MB/s eta 0:00:14
---------------------- ------------ 87.7/124.1 MB 2.8 MB/s eta 0:00:14
---------------------- ------------ 87.9/124.1 MB 2.7 MB/s eta 0:00:14
---------------------- ------------ 88.2/124.1 MB 2.7 MB/s eta 0:00:14
---------------------- ------------ 88.4/124.1 MB 2.7 MB/s eta 0:00:14
---------------------- ------------ 88.6/124.1 MB 2.7 MB/s eta 0:00:14
---------------------- ------------ 88.7/124.1 MB 2.7 MB/s eta 0:00:14
---------------------- ------------ 88.9/124.1 MB 2.7 MB/s eta 0:00:13
---------------------- ------------ 89.1/124.1 MB 2.7 MB/s eta 0:00:13
---------------------- ------------ 89.2/124.1 MB 2.7 MB/s eta 0:00:13
---------------------- ------------ 89.3/124.1 MB 2.8 MB/s eta 0:00:13
---------------------- ------------ 89.5/124.1 MB 2.8 MB/s eta 0:00:13
---------------------- ------------ 89.6/124.1 MB 2.8 MB/s eta 0:00:13
---------------------- ------------ 89.8/124.1 MB 2.8 MB/s eta 0:00:13
---------------------- ----------- 90.1/124.1 MB 2.8 MB/s eta 0:00:13
---------------------- ----------- 90.2/124.1 MB 2.8 MB/s eta 0:00:13
```

```
---------------------------- ---------- 90.3/124.1 MB 2.8 MB/s eta 0:00:12
---------------------------- ---------- 90.5/124.1 MB 2.9 MB/s eta 0:00:12
---------------------------- ---------- 90.6/124.1 MB 2.8 MB/s eta 0:00:12
---------------------------- ---------- 90.9/124.1 MB 2.8 MB/s eta 0:00:12
---------------------------- ---------- 91.2/124.1 MB 2.8 MB/s eta 0:00:12
---------------------------- ---------- 91.3/124.1 MB 2.9 MB/s eta 0:00:12
---------------------------- ---------- 91.6/124.1 MB 2.9 MB/s eta 0:00:12
---------------------------- ---------- 91.8/124.1 MB 2.9 MB/s eta 0:00:12
---------------------------- ---------- 92.0/124.1 MB 2.9 MB/s eta 0:00:11
---------------------------- ---------- 92.2/124.1 MB 2.9 MB/s eta 0:00:11
---------------------------- ---------- 92.4/124.1 MB 2.9 MB/s eta 0:00:11
---------------------------- ---------- 92.6/124.1 MB 3.0 MB/s eta 0:00:11
---------------------------- ---------- 92.8/124.1 MB 3.0 MB/s eta 0:00:11
---------------------------- ---------- 92.9/124.1 MB 3.0 MB/s eta 0:00:11
----------------------------- --------- 93.1/124.1 MB 3.0 MB/s eta 0:00:11
----------------------------- --------- 93.3/124.1 MB 3.0 MB/s eta 0:00:11
----------------------------- --------- 93.5/124.1 MB 3.0 MB/s eta 0:00:11
----------------------------- --------- 93.7/124.1 MB 3.0 MB/s eta 0:00:11
----------------------------- --------- 93.9/124.1 MB 3.0 MB/s eta 0:00:11
----------------------------- --------- 94.1/124.1 MB 3.0 MB/s eta 0:00:11
----------------------------- --------- 94.2/124.1 MB 3.0 MB/s eta 0:00:10
----------------------------- --------- 94.5/124.1 MB 3.0 MB/s eta 0:00:10
----------------------------- --------- 94.6/124.1 MB 3.0 MB/s eta 0:00:10
----------------------------- --------- 94.8/124.1 MB 3.0 MB/s eta 0:00:10
----------------------------- --------- 94.9/124.1 MB 3.0 MB/s eta 0:00:10
----------------------------- --------- 95.0/124.1 MB 3.0 MB/s eta 0:00:10
----------------------------- --------- 95.2/124.1 MB 3.1 MB/s eta 0:00:10
----------------------------- --------- 95.3/124.1 MB 3.1 MB/s eta 0:00:10
----------------------------- --------- 95.5/124.1 MB 3.1 MB/s eta 0:00:10
----------------------------- --------- 95.7/124.1 MB 3.1 MB/s eta 0:00:10
----------------------------- --------- 95.9/124.1 MB 3.1 MB/s eta 0:00:09
------------------------------ -------- 96.2/124.1 MB 3.2 MB/s eta 0:00:09
------------------------------ -------- 96.3/124.1 MB 3.3 MB/s eta 0:00:09
------------------------------ -------- 96.6/124.1 MB 3.3 MB/s eta 0:00:09
------------------------------ -------- 96.7/124.1 MB 3.3 MB/s eta 0:00:09
------------------------------ -------- 96.8/124.1 MB 3.3 MB/s eta 0:00:09
------------------------------ -------- 96.9/124.1 MB 3.2 MB/s eta 0:00:09
------------------------------ -------- 97.2/124.1 MB 3.3 MB/s eta 0:00:09
------------------------------ -------- 97.3/124.1 MB 3.2 MB/s eta 0:00:09
------------------------------ -------- 97.4/124.1 MB 3.2 MB/s eta 0:00:09
------------------------------ -------- 97.7/124.1 MB 3.3 MB/s eta 0:00:09
------------------------------ -------- 97.9/124.1 MB 3.3 MB/s eta 0:00:09
------------------------------ -------- 98.1/124.1 MB 3.3 MB/s eta 0:00:08
------------------------------ -------- 98.4/124.1 MB 3.3 MB/s eta 0:00:08
------------------------------ -------- 98.7/124.1 MB 3.3 MB/s eta 0:00:08
------------------------------ -------- 98.9/124.1 MB 3.3 MB/s eta 0:00:08
------------------------------ -------- 99.0/124.1 MB 3.3 MB/s eta 0:00:08
------------------------------ -------- 99.1/124.1 MB 3.3 MB/s eta 0:00:08
------------------------------- ------- 99.4/124.1 MB 3.4 MB/s eta 0:00:08
------------------------------- ------- 99.6/124.1 MB 3.3 MB/s eta 0:00:08
------------------------------- ------- 99.7/124.1 MB 3.3 MB/s eta 0:00:08
------------------------------- ------- 100.0/124.1 MB 3.4 MB/s eta 0:00:08
------------------------------- ------- 100.1/124.1 MB 3.4 MB/s eta 0:00:08
------------------------------- ------- 100.3/124.1 MB 3.4 MB/s eta 0:00:08
------------------------------- ------- 100.5/124.1 MB 3.4 MB/s eta 0:00:07
------------------------------- ------- 100.6/124.1 MB 3.4 MB/s eta 0:00:07
------------------------------- ------- 100.7/124.1 MB 3.4 MB/s eta 0:00:07
------------------------------- ------- 100.9/124.1 MB 3.4 MB/s eta 0:00:07
------------------------------- ------- 101.1/124.1 MB 3.3 MB/s eta 0:00:07
------------------------------- ------- 101.2/124.1 MB 3.3 MB/s eta 0:00:07
------------------------------- ------- 101.5/124.1 MB 3.3 MB/s eta 0:00:07
------------------------------- ------- 101.6/124.1 MB 3.3 MB/s eta 0:00:07
------------------------------- ------- 101.7/124.1 MB 3.3 MB/s eta 0:00:07
------------------------------- ------- 102.1/124.1 MB 3.3 MB/s eta 0:00:07
------------------------------- ------- 102.2/124.1 MB 3.3 MB/s eta 0:00:07
-------------------------------- ------ 102.5/124.1 MB 3.3 MB/s eta 0:00:07
-------------------------------- ------ 102.6/124.1 MB 3.3 MB/s eta 0:00:07
-------------------------------- ------ 102.8/124.1 MB 3.3 MB/s eta 0:00:07
-------------------------------- ------ 103.0/124.1 MB 3.3 MB/s eta 0:00:07
-------------------------------- ------ 103.2/124.1 MB 3.4 MB/s eta 0:00:07
-------------------------------- ------ 103.3/124.1 MB 3.3 MB/s eta 0:00:07
```

```
-------------------------------- ------ 103.5/124.1 MB 3.3 MB/s eta 0:00:07
-------------------------------- ------ 103.9/124.1 MB 3.3 MB/s eta 0:00:07
-------------------------------- ------ 104.0/124.1 MB 3.3 MB/s eta 0:00:07
-------------------------------- ------ 104.2/124.1 MB 3.3 MB/s eta 0:00:06
-------------------------------- ------ 104.4/124.1 MB 3.3 MB/s eta 0:00:06
-------------------------------- ------ 104.5/124.1 MB 3.3 MB/s eta 0:00:06
-------------------------------- ------ 104.7/124.1 MB 3.3 MB/s eta 0:00:06
-------------------------------- ------ 104.9/124.1 MB 3.3 MB/s eta 0:00:06
-------------------------------- ------ 105.1/124.1 MB 3.3 MB/s eta 0:00:06
-------------------------------- ------ 105.2/124.1 MB 3.3 MB/s eta 0:00:06
--------------------------------- ----- 105.8/124.1 MB 3.4 MB/s eta 0:00:06
--------------------------------- ----- 106.1/124.1 MB 3.4 MB/s eta 0:00:06
--------------------------------- ----- 106.4/124.1 MB 3.4 MB/s eta 0:00:06
--------------------------------- ----- 106.6/124.1 MB 3.4 MB/s eta 0:00:06
--------------------------------- ----- 106.8/124.1 MB 3.4 MB/s eta 0:00:06
--------------------------------- ----- 106.9/124.1 MB 3.4 MB/s eta 0:00:06
--------------------------------- ----- 107.0/124.1 MB 3.4 MB/s eta 0:00:05
--------------------------------- ----- 107.6/124.1 MB 3.5 MB/s eta 0:00:05
--------------------------------- ----- 107.8/124.1 MB 3.5 MB/s eta 0:00:05
--------------------------------- ----- 108.0/124.1 MB 3.6 MB/s eta 0:00:05
--------------------------------- ----- 108.4/124.1 MB 3.5 MB/s eta 0:00:05
--------------------------------- ----- 108.5/124.1 MB 3.6 MB/s eta 0:00:05
---------------------------------- ---- 108.8/124.1 MB 3.5 MB/s eta 0:00:05
---------------------------------- ---- 108.9/124.1 MB 3.5 MB/s eta 0:00:05
---------------------------------- ---- 109.3/124.1 MB 3.6 MB/s eta 0:00:05
---------------------------------- ---- 109.4/124.1 MB 3.6 MB/s eta 0:00:05
---------------------------------- ---- 109.5/124.1 MB 3.6 MB/s eta 0:00:05
---------------------------------- ---- 109.6/124.1 MB 3.5 MB/s eta 0:00:05
---------------------------------- ---- 109.8/124.1 MB 3.5 MB/s eta 0:00:05
---------------------------------- ---- 110.4/124.1 MB 3.6 MB/s eta 0:00:04
---------------------------------- ---- 110.6/124.1 MB 3.6 MB/s eta 0:00:04
---------------------------------- ---- 110.8/124.1 MB 3.6 MB/s eta 0:00:04
---------------------------------- ---- 111.2/124.1 MB 3.7 MB/s eta 0:00:04
---------------------------------- ---- 111.4/124.1 MB 3.7 MB/s eta 0:00:04
---------------------------------- ---- 111.6/124.1 MB 3.7 MB/s eta 0:00:04
----------------------------------- --- 111.8/124.1 MB 3.7 MB/s eta 0:00:04
----------------------------------- --- 111.9/124.1 MB 3.8 MB/s eta 0:00:04
----------------------------------- --- 112.5/124.1 MB 3.7 MB/s eta 0:00:04
----------------------------------- --- 112.5/124.1 MB 3.7 MB/s eta 0:00:04
----------------------------------- --- 112.9/124.1 MB 3.8 MB/s eta 0:00:03
----------------------------------- --- 113.0/124.1 MB 3.8 MB/s eta 0:00:03
----------------------------------- --- 113.3/124.1 MB 3.7 MB/s eta 0:00:03
----------------------------------- --- 113.6/124.1 MB 3.8 MB/s eta 0:00:03
----------------------------------- --- 113.8/124.1 MB 3.9 MB/s eta 0:00:03
----------------------------------- --- 114.1/124.1 MB 3.9 MB/s eta 0:00:03
----------------------------------- --- 114.5/124.1 MB 3.9 MB/s eta 0:00:03
----------------------------------- --- 114.7/124.1 MB 3.9 MB/s eta 0:00:03
------------------------------------ -- 114.9/124.1 MB 3.9 MB/s eta 0:00:03
------------------------------------ -- 115.1/124.1 MB 4.0 MB/s eta 0:00:03
------------------------------------ -- 115.6/124.1 MB 4.1 MB/s eta 0:00:03
------------------------------------ -- 115.8/124.1 MB 4.0 MB/s eta 0:00:03
------------------------------------ -- 116.0/124.1 MB 4.1 MB/s eta 0:00:02
------------------------------------ -- 116.2/124.1 MB 4.1 MB/s eta 0:00:02
------------------------------------ -- 116.3/124.1 MB 4.1 MB/s eta 0:00:02
------------------------------------ -- 116.7/124.1 MB 4.1 MB/s eta 0:00:02
------------------------------------ -- 116.8/124.1 MB 4.1 MB/s eta 0:00:02
------------------------------------ -- 116.8/124.1 MB 4.1 MB/s eta 0:00:02
------------------------------------ -- 117.0/124.1 MB 4.0 MB/s eta 0:00:02
------------------------------------ -- 117.2/124.1 MB 4.0 MB/s eta 0:00:02
------------------------------------ -- 117.5/124.1 MB 4.2 MB/s eta 0:00:02
------------------------------------ -- 117.8/124.1 MB 4.1 MB/s eta 0:00:02
------------------------------------- - 118.1/124.1 MB 4.1 MB/s eta 0:00:02
------------------------------------- - 118.1/124.1 MB 4.0 MB/s eta 0:00:02
------------------------------------- - 118.2/124.1 MB 4.0 MB/s eta 0:00:02
------------------------------------- - 118.8/124.1 MB 4.1 MB/s eta 0:00:02
------------------------------------- - 119.0/124.1 MB 4.1 MB/s eta 0:00:02
------------------------------------- - 119.1/124.1 MB 4.2 MB/s eta 0:00:02
------------------------------------- - 119.4/124.1 MB 4.2 MB/s eta 0:00:02
------------------------------------- - 119.8/124.1 MB 4.3 MB/s eta 0:00:02
------------------------------------- - 120.1/124.1 MB 4.4 MB/s eta 0:00:01
------------------------------------- - 120.3/124.1 MB 4.5 MB/s eta 0:00:01
```

```
   ------------------------------------ - 120.6/124.1 MB 4.4 MB/s eta 0:00:01
   ------------------------------------ - 120.8/124.1 MB 4.4 MB/s eta 0:00:01
   -------------------------------------  121.1/124.1 MB 4.5 MB/s eta 0:00:01
   -------------------------------------  121.4/124.1 MB 4.5 MB/s eta 0:00:01
   -------------------------------------  121.7/124.1 MB 4.5 MB/s eta 0:00:01
   -------------------------------------  121.9/124.1 MB 4.5 MB/s eta 0:00:01
   -------------------------------------  122.2/124.1 MB 4.7 MB/s eta 0:00:01
   -------------------------------------  122.7/124.1 MB 4.6 MB/s eta 0:00:01
   -------------------------------------  122.9/124.1 MB 4.7 MB/s eta 0:00:01
   -------------------------------------  123.2/124.1 MB 4.7 MB/s eta 0:00:01
   -------------------------------------  123.4/124.1 MB 4.7 MB/s eta 0:00:01
   -------------------------------------  123.6/124.1 MB 4.7 MB/s eta 0:00:01
   -------------------------------------  124.1/124.1 MB 4.8
```

```
   ---------------------------- ---------- 9.6/12.8 MB 4.9 MB/s eta 0:00:01
   ----------------------------- --------- 9.9/12.8 MB 5.0 MB/s eta 0:00:01
   ------------------------------ ------- 10.4/12.8 MB 5.0 MB/s eta 0:00:01
   ------------------------------- ------ 10.8/12.8 MB 5.0 MB/s eta 0:00:01
   -------------------------------- ---- 11.3/12.8 MB 5.2 MB/s eta 0:00:01
   --------------------------------- --- 11.8/12.8 MB 5.2 MB/s eta 0:00:01
   ---------------------------------- -- 12.1/12.8 MB 5.5 MB/s eta 0:00:01
   ---------------------------------- - 12.5/12.8 MB 5.5 MB/s eta 0:00:01
   ----------------------------------  12.8/12.8 MB 5.5 MB/s eta 0:00:01
   ----------------------------------  12.8/12.8 MB 5.4 MB/s eta 0:00:01
   ----------------------------------  12.8/12.8 MB 5.4 MB/s eta 0:00:01
   ----------------------------------  12.8/12.8 MB 5.4 MB/s eta 0:00:01
   ----------------------------------  12.8/12.8 MB 5.4 MB/s eta 0:00:01
   ---------------------------------- 12.8/12.8 MB 4.9 MB/s eta 0:00:00
Downloading fsspec-2026.9.0-py3-none-any.whl (221 kB)
   ---------------------------------------- 0.0/221.7 kB ? eta -:--:--
   ---------------------------------- - 215.0/221.7 kB 12.8 MB/s eta 0:00:01
   ---------------------------------------- 221.7/221.7 kB 3.4 MB/s eta 0:00:00
Downloading huggingface_hub-2.2.0-py3-none-any.whl (839 kB)
   ---------------------------------------- 0.0/839.9 kB ? eta -:--:--
   ----------------- ---------------------- 378.9/839.9 kB 7.8 MB/s eta 0:00:01
   ------------------------------- -------- 665.6/839.9 kB 7.0 MB/s eta 0:00:01
   ---------------------------------------- 839.9/839.9 kB 4.4 MB/s eta 0:00:00
Downloading filelock-4.2.4-py3-none-any.whl (162 kB)
   ---------------------------------------- 0.0/162.5 kB ? eta -:--:--
   ---------------------------------- -- 153.6/162.5 kB 9.0 MB/s eta 0:00:01
   ---------------------------------------- 162.5/162.5 kB 2.0 MB/s eta 0:00:00
Downloading networkx-3.7-py3-none-any.whl (2.1 MB)
   ---------------------------------------- 0.0/2.1 MB ? eta -:--:--
   ----------- ---------------------------- 0.6/2.1 MB 13.3 MB/s eta 0:00:01
   ------------------ --------------------- 1.0/2.1 MB 11.0 MB/s eta 0:00:01
   ------------------------- -------------- 1.4/2.1 MB 10.0 MB/s eta 0:00:01
   ---------------------------------- ------ 1.8/2.1 MB 8.8 MB/s eta 0:00:01
   ---------------------------------------  2.1/2.1 MB 9.1 MB/s eta 0:00:01
   ---------------------------------------  2.1/2.1 MB 9.1 MB/s eta 0:00:01
   ---------------------------------------- 2.1/2.1 MB 6.5 MB/s eta 0:00:00
Downloading safetensors-0.8.0-cp310-abi3-win_amd64.whl (355 kB)
   ---------------------------------------- 0.0/355.5 kB ? eta -:--:--
   ---------------------------------------  348.2/355.5 kB 10.6 MB/s eta 0:00:01
   ---------------------------------------- 355.5/355.5 kB 4.4 MB/s eta 0:00:00
Using cached sympy-1.14.0-py3-none-any.whl (6.3 MB)
Downloading tokenizers-0.23.3-cp310-abi3-win_amd64.whl (2.9 MB)
   ---------------------------------------- 0.0/2.9 MB ? eta -:--:--
   ------- -------------------------------- 0.5/2.9 MB 16.3 MB/s eta 0:00:01
   ------------- -------------------------- 0.9/2.9 MB 9.9 MB/s eta 0:00:01
   ------------------- -------------------- 1.4/2.9 MB 10.0 MB/s eta 0:00:01
   ------------------------ --------------- 1.7/2.9 MB 8.3 MB/s eta 0:00:01
   ----------------------------- ---------- 2.1/2.9 MB 8.0 MB/s eta 0:00:01
   ---------------------------------- ----- 2.4/2.9 MB 7.8
```

```
   ---------------------------------------- 0.0/97.9 kB ? eta -:--:--
   ---------------------------------------- 97.9/97.9 kB 1.9 MB/s eta 0:00:00
Downloading httpcore2-2.16.0-py3-none-any.whl (85 kB)
   ---------------------------------------- 0.0/85.1 kB ? eta -:--:--
   ---------------------------------------- - 81.9/85.1 kB ? eta -:--:--
   ---------------------------------------- 85.1/85.1 kB 1.2 MB/s eta 0:00:00
Using cached mpmath-1.3.0-py3-none-any.whl (536 kB)
Downloading truststore-0.10.5-py3-none-any.whl (19 kB)
Installing collected packages: mpmath, truststore, sympy, safetensors, networkx, hf-xe
t, fsspec, filelock, torch, httpcore2, httpx2, huggingface-hub, tokenizers, transformer
s
Successfully installed filelock-4.2.4 fsspec-2026.9.0 hf-xet-1.7.0 httpcore2-2.16.0 htt
px2-2.16.0 huggingface-hub-2.2.0 mpmath-1.3.0 networkx-3.7 safetensors-0.8.0 sympy-1.1
4.0 tokenizers-0.23.3 torch-2.14.1 transformers-5.20.2 truststore-0.10.5
Note: you may need to restart the kernel to use updated packages.

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
to enable higher rate limits and faster downloads.

config.json:   0%|          | 0.00/570 [00:00<?, ?B/s]

tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]

vocab.txt:   0%|          | 0.00/232k [00:00<?, ?B/s]

tokenizer.json:   0%|          | 0.00/466k [00:00<?, ?B/s]

model.safetensors: reconstructing file:   0%|          |  0.00B /  440MB

model.safetensors: downloading bytes:           |  0.00B

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]

[transformers] BertModel LOAD REPORT from: bert-base-uncased
Key                                         | Status     |  |
--------------------------------------------+------------+--+-
cls.seq_relationship.bias                   | UNEXPECTED |  |
cls.predictions.transform.dense.weight      | UNEXPECTED |  |
cls.predictions.transform.dense.bias        | UNEXPECTED |  |
cls.seq_relationship.weight                 | UNEXPECTED |  |
cls.predictions.bias                        | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |  |

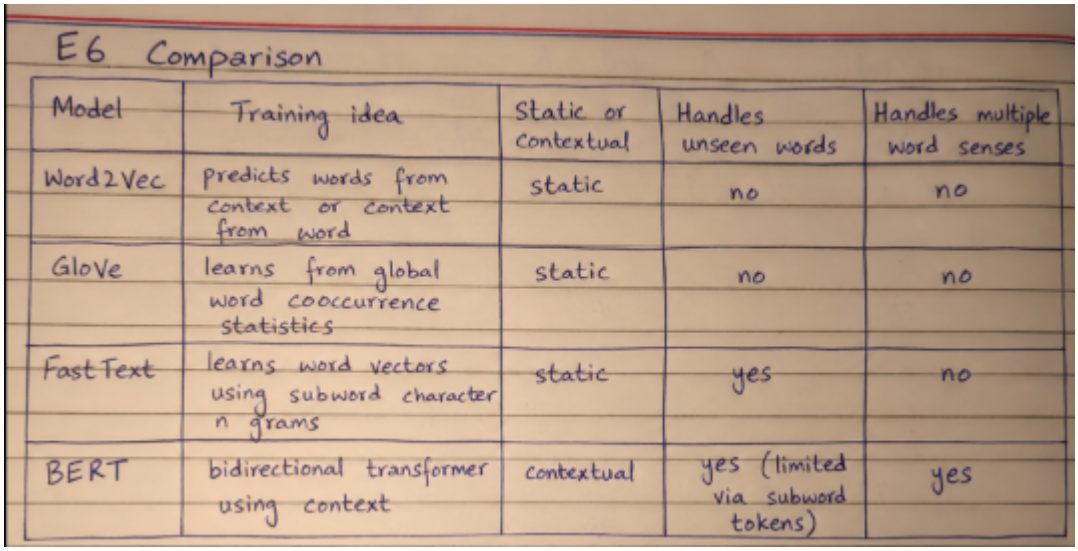
Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if
you expect identical arch.

Sentence 1 vs 2: 0.5191
Sentence 1 vs 3: 0.4359
Sentence 2 vs 3: 0.885
```

Explanation of Results

glove produces the same vector for bank in every sentence because it uses static word embeddings bert produces different vectors because it considers the surrounding words and understands the context in which bank is used in the first sentence bank refers to the side of a river while in the second and third sentences bank refers to a financial institution the second and third sentences are expected to have higher similarity because both use bank in a financial context the cosine similarity values show how closely the contextual representations are related

E6



E6 Comparison

Model	Training idea	Static or contextual	Handles unseen words	Handles multiple word senses
Word2Vec	predicts words from context or context from word	static	no	no
GloVe	learns from global word cooccurrence statistics	static	no	no
FastText	learns word vectors using subword character n grams	static	yes	no
BERT	bidirectional transformer using context	contextual	yes (limited via subword tokens)	yes

F1

In [52]:

```
from sklearn.datasets import fetch_20newsgroups

data = fetch_20newsgroups(
    categories=["sci.space", "rec.autos"],
    remove=("headers", "footers", "quotes")
)

texts = data.data[:500]
labels = np.array(data.target[:500])

# Load spaCy model
nlp = spacy.load("en_core_web_sm", disable=["parser", "ner"])

def process_text(text):
    text = text.lower()
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"[^a-z\s]", " ", text)

    doc = nlp(text)

    return [t.lemma_ for t in doc if t.is_alpha and not t.is_stop]

before = [len(nlp.make_doc(t)) for t in texts]

processed = [process_text(t) for t in texts]

after = [len(t) for t in processed]

print("Documents Processed:", len(texts))
print("Average Tokens Before:", round(np.mean(before), 2))
print("Average Tokens After:", round(np.mean(after), 2))
```

```
Documents Processed: 500
Average Tokens Before: 243.75
Average Tokens After: 95.33
```

Explanation of Results

the pipeline processes 500 documents from the selected categories regular expressions remove unwanted characters and spacy performs tokenization stop word removal and lemmatization the average number of tokens decreases after processing because unnecessary words and characters are removed

F2

In [53]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
docs = [" ".join(p) for p in processed]
labels = np.array(data.target[:500])

tfidf = TfidfVectorizer(ngram_range=(1, 2), max_features=5000)
X = tfidf.fit_transform(docs)
terms = np.array(tfidf.get_feature_names_out())
print("TF-IDF matrix:", X.shape)

for c, name in enumerate(data.target_names):
    mean_scores = np.asarray(X[labels == c].mean(axis=0)).ravel()
    top = terms[mean_scores.argsort()[::-1][:10]]
    print(f"{name:<10}", list(top))
```

```
TF-IDF matrix: (500, 5000)
rec.autos  ['car', 'drive', 'like', 'new', 'think', 'good', 'buy', 'know', 'right', 'lo
ok']
sci.space  ['space', 'launch', 'nasa', 'think', 'orbit', 'know', 'like', 'moon', 'satel
lite', 'thing']
```

Explanation of Results

the top terms for rec autos are expected to relate to cars engines vehicles and driving because the documents discuss automobiles the top terms for sci space are expected to relate to space satellites nasa and scientific research because the documents discuss space topics the results make sense when the highest ranked terms are related to their respective categories because tf idf gives importance to distinctive words

F3

Reflection

TF-IDF identifies important words but cannot understand their meanings or fully capture word order and context. For example, it treats *car* and *automobile* as different features. **GloVe embeddings** capture semantic similarities but lose context when word vectors are averaged.

BERT captures word meaning based on surrounding context and may improve classification, but it requires more memory and computational power.

In [ ]:
```
```